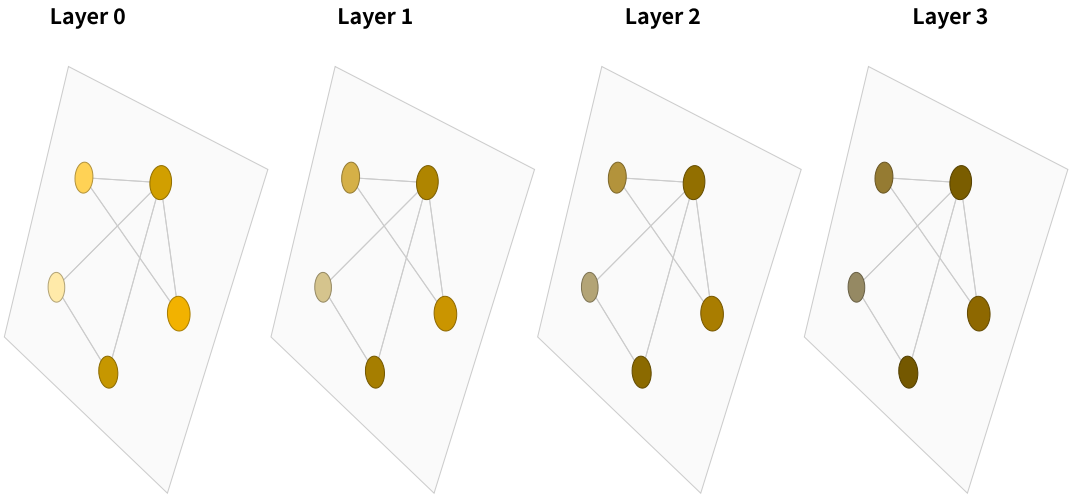
A Gentle Introduction to Graph Neural Networks

Neural networks have been adapted to leverage the structure and properties of graphs. We explore the components needed for building a graph neural network - and motivate the design choices behind them.



Hover over a node in the diagram below to see how it accumulates information from nodes around it through the layers of the network.

AUTHORS	AFFILIATIONS
Benjamin Sanchez-Lengeling	Google Research
Emily Reif	Google Research
Adam Pearce	Google Research
Alexander B. Wiltschko	Google Research

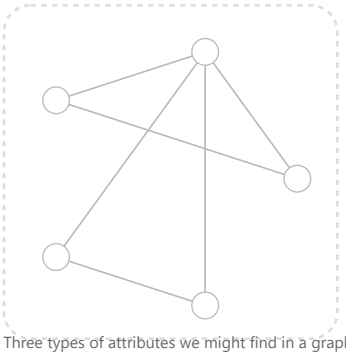
PUBLISHED	DOI
Sept. 2, 2021	10.23915/distill.00033

This article is one of two Distill publications about graph neural networks. Take a look at Understanding Convolutions on Graphs [1] to understand how convolutions over images generalize naturally to convolutions over graphs.

Graphs are all around us; real world objects are often defined in terms of their connections to other things. A set of objects, and the connections between them, are naturally expressed as a *graph*. Researchers have developed neural networks that operate on graph data (called graph neural networks, or GNNs) for over a decade [2]. Recent developments have increased their capabilities and expressive power. We are starting to see practical applications in areas such as antibacterial discovery [3], physics simulations [4], fake news detection [5], traffic prediction [6] and recommendation systems [7].

This article explores and explains modern graph neural networks. We divide this work into four parts. First, we look at what kind of data is most naturally phrased as a graph, and some common examples. Second, we explore what makes graphs different from other types of data, and some of the specialized choices we have to make when using graphs. Third, we build a modern GNN, walking through each of the parts of the model, starting with historic modeling innovations in the field. We move gradually from a bare-bones implementation to a state-of-the-art GNN model. Fourth and finally, we provide a GNN playground where you can play around with a real-word task and dataset to build a stronger intuition of how each component of a GNN model contributes to the predictions it makes.

To start, let's establish what a graph is. A graph represents the relations (*edges*) between a collection of entities (*nodes*).



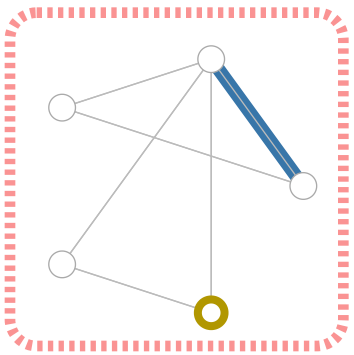
V Vertex (or node) attributes
e.g., node identity, number of neighbors

E Edge (or link) attributes and directions
e.g., edge identity, edge weight

U Global (or master node) attributes
e.g., number of nodes, longest path

Three types of attributes we might find in a graph, hover over to highlight each attribute. Other types of graphs and attributes are explored in the Other types of graphs section.

To further describe each node, edge or the entire graph, we can store information in each of these pieces of the graph.



Vertex (or node) embedding

Edge (or link) attributes and embedding

Global (or master node) embedding

Information in the form of scalars or embeddings can be stored at each graph node (left) or edge (right).

We can additionally specialize graphs by associating directionality to edges (*directed, undirected*).



The edges can be directed, where an edge e has a source node, v_{src}, and a destination node v_{dst}. In this case, information flows from v_{src} to v_{dst}. They can also be undirected, where there is no notion of source or destination nodes, and information flows both directions. Note that having a single undirected edge is equivalent to having one directed edge from v_{src} to v_{dst}, and another directed edge from v_{dst} to v_{src}.

Graphs are very flexible data structures, and if this seems abstract now, we will make it concrete with examples in the next section.

Graphs and where to find them

You're probably already familiar with some types of graph data, such as social networks. However, graphs are an extremely powerful and general representation of data, we will show two types of data that you might not think could be modeled as graphs: images and text. Although counterintuitive, one can learn more about the symmetries and structure of images and text by viewing them as graphs,, and build an intuition that will help understand other less grid-like graph data, which we will discuss later.

Images as graphs

We typically think of images as rectangular grids with image channels, representing them as arrays (e.g., 244x244x3 floats). Another way to think of images is as graphs with regular structure, where each pixel represents a node and is connected via an edge to adjacent pixels. Each non-border pixel has exactly 8 neighbors, and the information stored at each node is a 3-dimensional vector representing the RGB value of the pixel.

A way of visualizing the connectivity of a graph is through its *adjacency matrix*. We order the nodes, in this case each of 25 pixels in a simple 5x5 image of a smiley face, and fill a matrix of $n_{nodes} \times n_{nodes}$ with an entry if two nodes share an edge. Note that each of these three representations below are different views of the same piece of data.

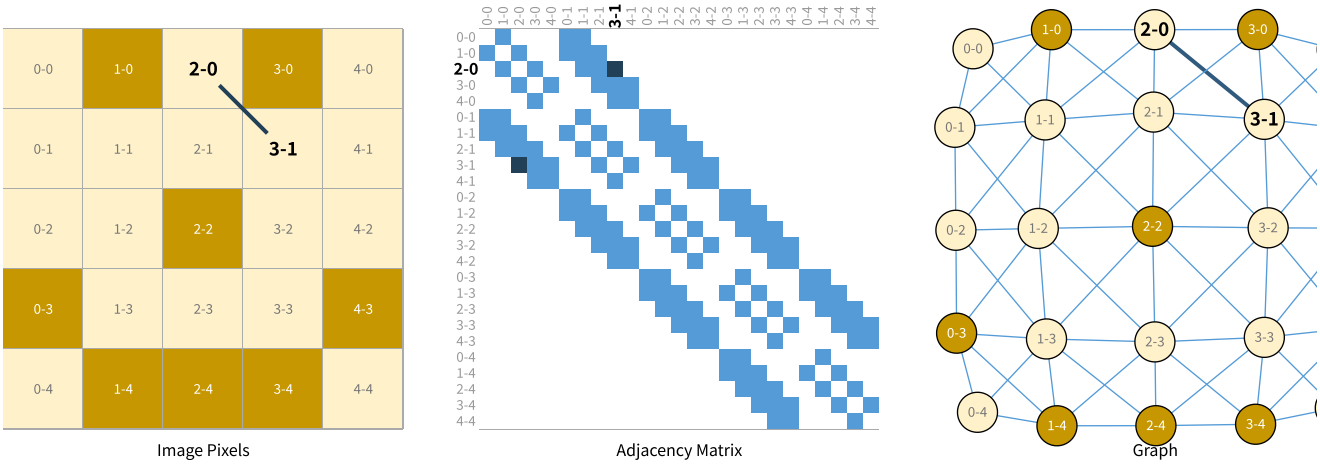


Image Pixels Adjacency Matrix Graph

Click on an image pixel to toggle its value, and see how the graph representation changes.

Text as graphs

We can digitize text by associating indices to each character, word, or token, and representing text as a sequence of these indices. This creates a simple directed graph, where each character or index is a node and is connected via an edge to the node that follows it.



Edit the text above to see how the graph representation changes.

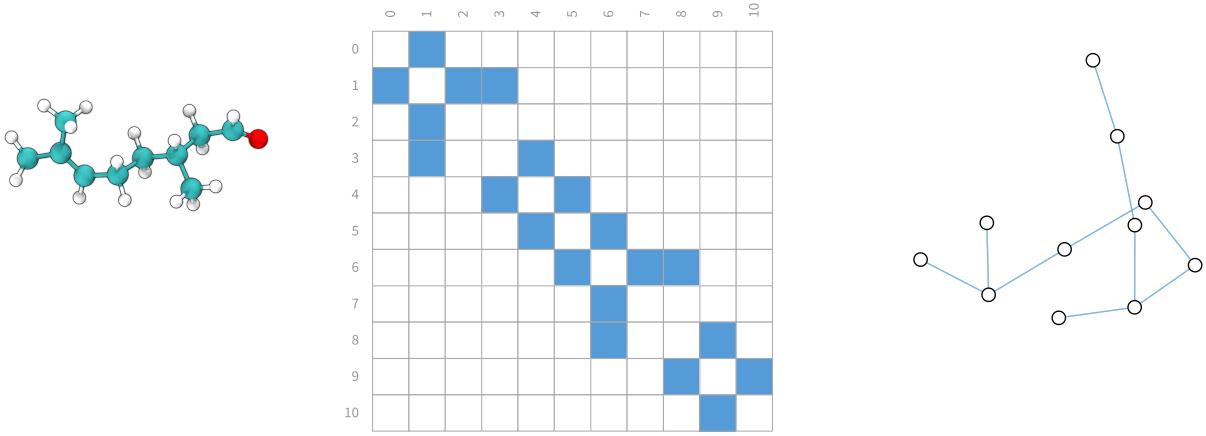
Of course, in practice, this is not usually how text and images are encoded: these graph representations are redundant since all images and all text will have very regular structures. For instance, images have a banded structure in their adjacency matrix because all nodes (pixels) are connected in a grid. The adjacency matrix for text is just a diagonal line, because each word only connects to the prior word, and to the next one.

This represe
sequence of
tokens) refer
text is often
RNNs; other
Transformers
considered t
fully connect
we learn the
between tok
in Graph Att
Networks.

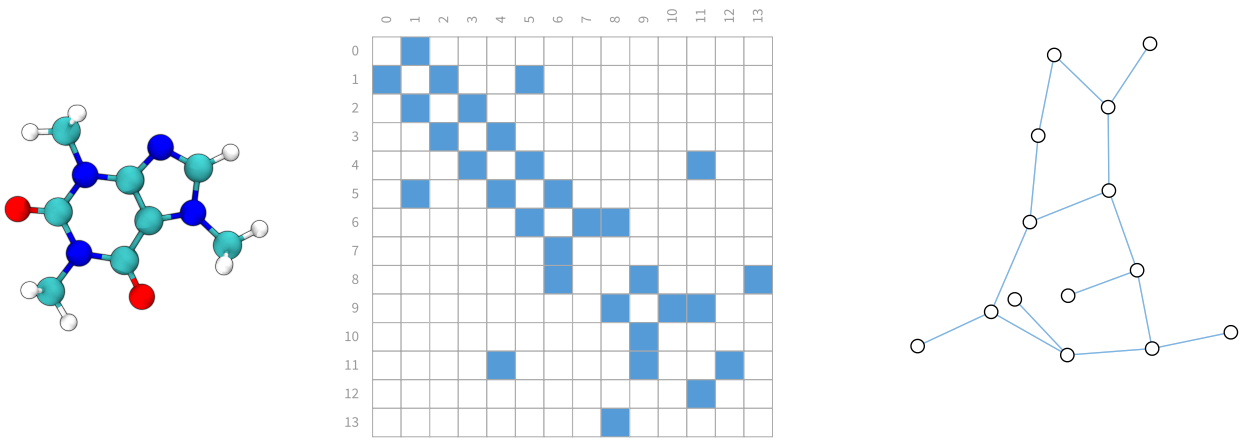
Graph-valued data in the wild

Graphs are a useful tool to describe data you might already be familiar with. Let's move on to data which is more heterogeneously structured. In these examples, the number of neighbors to each node is variable (as opposed to the fixed neighborhood size of images and text). This data is hard to phrase in any other way besides a graph.

Molecules as graphs. Molecules are the building blocks of matter, and are built of atoms and electrons in 3D space. All particles are interacting, but when a pair of atoms are stuck in a stable distance from each other, we say they share a covalent bond. Different pairs of atoms and bonds have different distances (e.g. single-bonds, double-bonds). It's a very convenient and common abstraction to describe this 3D object as a graph, where nodes are atoms and edges are covalent bonds. [8] Here are two common molecules, and their associated graphs.



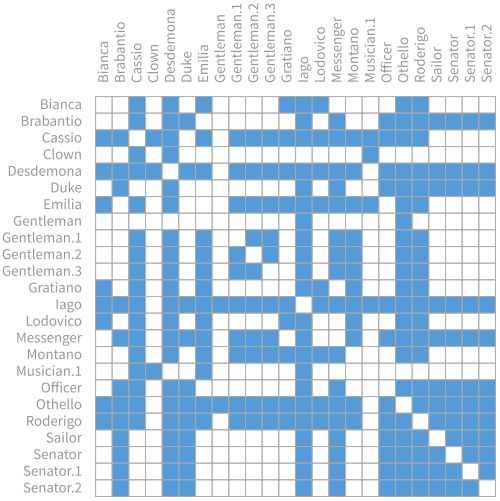
(Left) 3d representation of the Citronellal molecule (Center) Adjacency matrix of the bonds in the molecule (Right) Graph representation of the molecule.

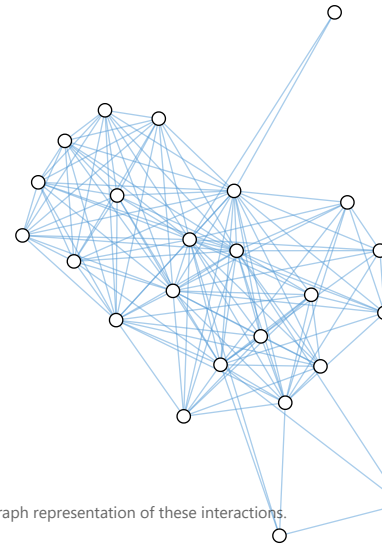


(Left) 3d representation of the Caffeine molecule (Center) Adjacency matrix of the bonds in the molecule (Right) Graph representation of the molecule.

Social networks as graphs. Social networks are tools to study patterns in collective behaviour of people, institutions and organizations. We can build a graph representing groups of people by modelling individuals as nodes, and their relationships as edges.



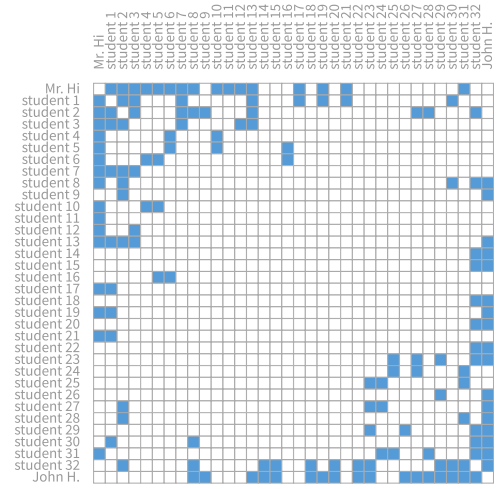


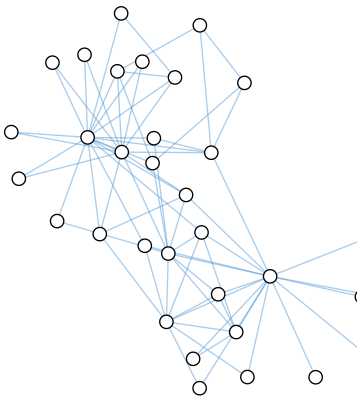


(Left) Image of a scene from the play "Othello". (Center) Adjacency matrix of the interaction between characters in the play. (Right) Graph representation of these interactions.

Unlike image and text data, social networks do not have identical adjacency matrices.







(Left) Image of karate tournament. (Center) Adjacency matrix of the interaction between people in a karate club. (Right) Graph representation of these interactions.

Citation networks as graphs. Scientists routinely cite other scientists' work when publishing papers. We can visualize these networks of citations as a graph, where each paper is a node, and each *directed* edge is a citation between one paper and another. Additionally, we can add information about each paper into each node, such as a word embedding of the abstract. (see [9], [10] , [11]).

Other examples. In computer vision, we sometimes want to tag objects in visual scenes. We can then build graphs by treating these objects as nodes, and their relationships as edges. Machine learning models, programming code [12] and math equations [13] can also be phrased as graphs, where the variables are nodes, and edges are operations that have these variables as input and output. You might see the term "dataflow graph" used in some of these contexts.

The structure of real-world graphs can vary greatly between different types of data — some graphs have many nodes with few connections between them, or vice versa. Graph datasets can vary widely (both within a given dataset, and between datasets) in terms of the number of nodes, edges, and the connectivity of nodes.

					Edges per node (degree)		
Dataset	**Domain**	**graphs**	**nodes**	**edges**	**min**	**mean**	**max**
karate club	Social network	1	34	78		4.5	17
qm9	Small molecules	134k	≤ 9	≤26	1	2	5
Cora	Citation network	1	23,166	91,500	1	7.8	379
Wikipedia links, English	Knowledge graph	1	12M	378M		62.24	1M

Summary statistics on graphs found in the real world. Numbers are dependent on featurization decisions. More useful statistics and graphs can be found in KONECT [14]

What types of problems have graph structured data?

We have described some examples of graphs in the wild, but what tasks do we want to perform on this data? There are three general types of prediction tasks on graphs: graph-level, node-level, and edge-level.

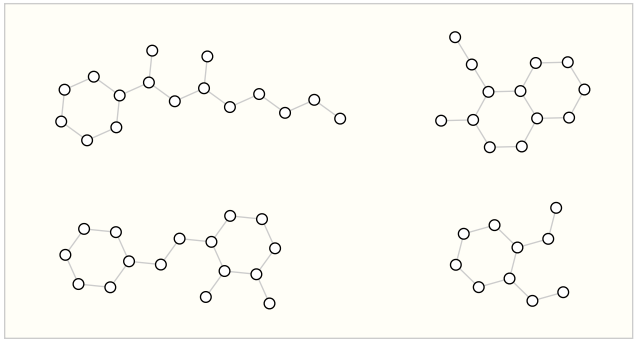
In a graph-level task, we predict a single property for a whole graph. For a node-level task, we predict some property for each node in a graph. For an edge-level task, we want to predict the property or presence of edges in a graph.

For the three levels of prediction problems described above (graph-level, node-level, and edge-level), we will show that all of the following problems can be solved with a single model class, the GNN. But first, let's take a tour through the three classes of graph prediction problems in more detail, and provide concrete examples of each.

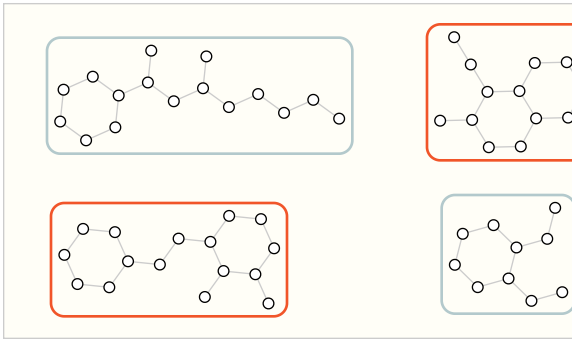
There are otl
that are area
research. For
might want t
graphs, or ex
predictions o
More topics
the Into the

Graph-level task

In a graph-level task, our goal is to predict the property of an entire graph. For example, for a molecule represented as a graph, we might want to predict what the molecule smells like, or whether it will bind to a receptor implicated in a disease.



Input: graphs



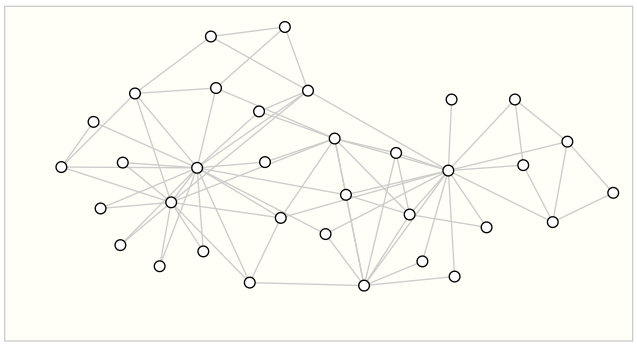
Output: labels for each graph, (e.g., "does the graph contain two ring

This is analogous to image classification problems with MNIST and CIFAR, where we want to associate a label to an entire image. With text, a similar problem is sentiment analysis where we want to identify the mood or emotion of an entire sentence at once.

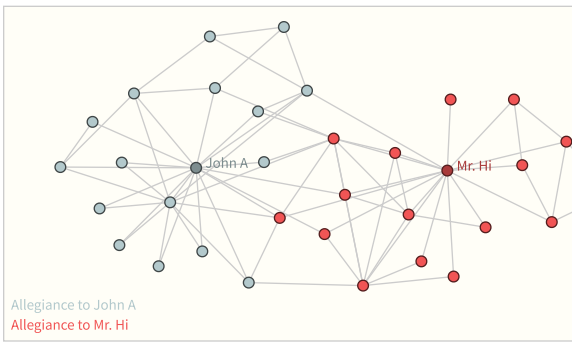
Node-level task

Node-level tasks are concerned with predicting the identity or role of each node within a graph.

A classic example of a node-level prediction problem is Zach's karate club.[15] The dataset is a single social network graph made up of individuals that have sworn allegiance to one of two karate clubs after a political rift. As the story goes, a feud between Mr. Hi (Instructor) and John H (Administrator) creates a schism in the karate club. The nodes represent individual karate practitioners, and the edges represent interactions between these members outside of karate. The prediction problem is to classify whether a given member becomes loyal to either Mr. Hi or John H, after the feud. In this case, distance between a node to either the Instructor or Administrator is highly correlated to this label.



Input: graph with unlabled nodes



Output: graph node labels

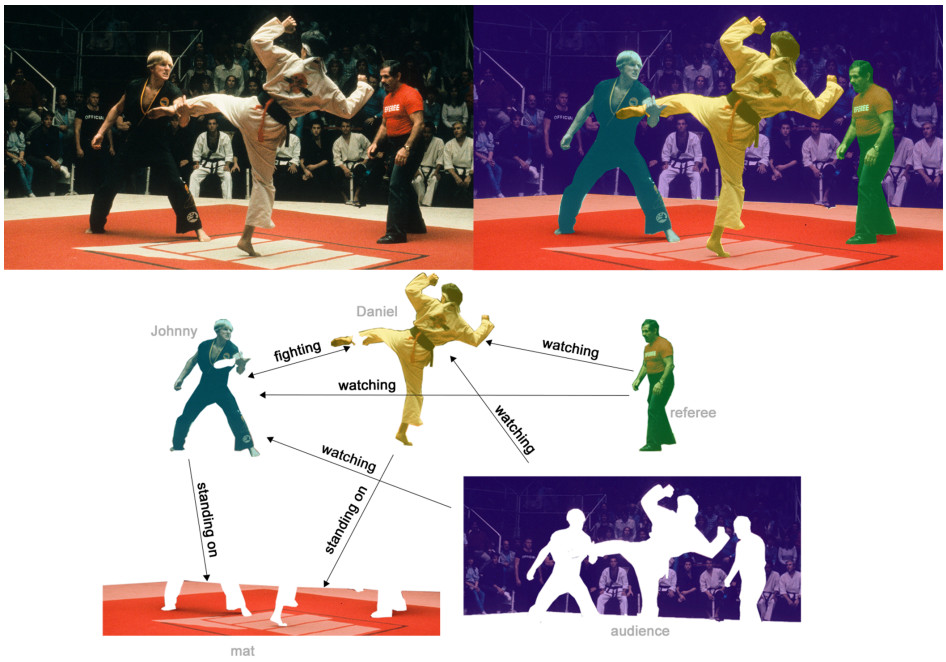
On the left we have the initial conditions of the problem, on the right we have a possible solution, where each node has been classified based on the alliance. The dataset can be used in other graph problems like unsupervised learning.

Following the image analogy, node-level prediction problems are analogous to *image segmentation*, where we are trying to label the role of each pixel in an image. With text, a similar task would be predicting the parts-of-speech of each word in a sentence (e.g. noun, verb, adverb, etc).

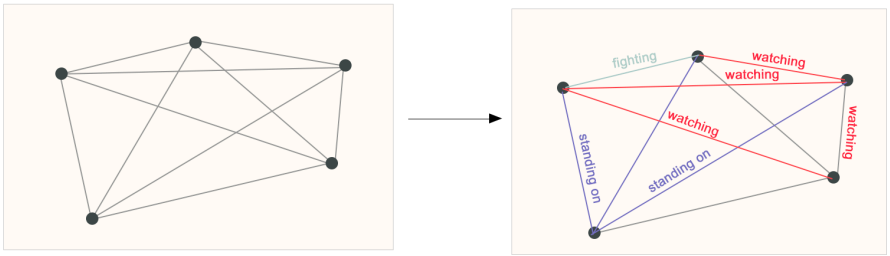
Edge-level task

The remaining prediction problem in graphs is *edge prediction*.

One example of edge-level inference is in image scene understanding. Beyond identifying objects in an image, deep learning models can be used to predict the relationship between them. We can phrase this as an edge-level classification: given nodes that represent the objects in the image, we wish to predict which of these nodes share an edge or what the value of that edge is. If we wish to discover connections between entities, we could consider the graph fully connected and based on their predicted value prune edges to arrive at a sparse graph.



In (b), above, the original image (a) has been segmented into five entities: each of the fighters, the referee, the audience and the mat. (C) shows the relationships between these entities.



On the left we have an initial graph built from the previous visual scene. On the right is a possible edge-labeling of this graph when some connections were pruned based on the model's output.

The challenges of using graphs in machine learning

So, how do we go about solving these different graph tasks with neural networks? The first step is to think about how we will represent graphs to be compatible with neural networks.

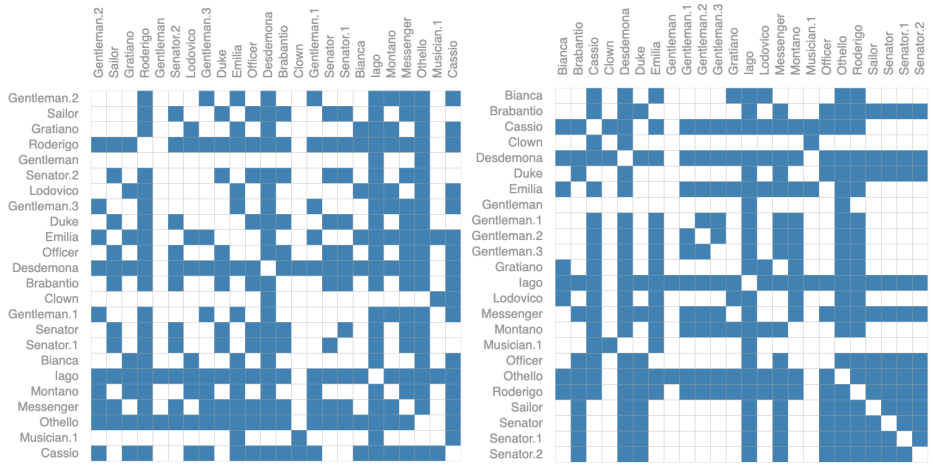
Machine learning models typically take rectangular or grid-like arrays as input. So, it's not immediately intuitive how to represent them in a format that is compatible with deep learning. Graphs have up to four types of information that we will potentially want to use to make predictions: nodes, edges, global-context and connectivity. The first three are relatively straightforward: for example, with nodes we can form a node feature matrix N by assigning each node an index i and storing the feature for $node_i$ in N. While these matrices have a variable number of examples, they can be processed without any special techniques.

However, representing a graph's connectivity is more complicated. Perhaps the most obvious choice would be to use an adjacency matrix, since this is easily tensorisable. However, this representation has a few drawbacks. From the example dataset table, we see the number of nodes in a graph can be on the order of millions, and the number of edges per node can be highly variable. Often, this leads to very sparse adjacency matrices, which are space-inefficient.

Another problem is that there are many adjacency matrices that can encode the same connectivity, and there is no guarantee that these different matrices would produce the same result in a deep neural network (that is to say, they are not permutation invariant).

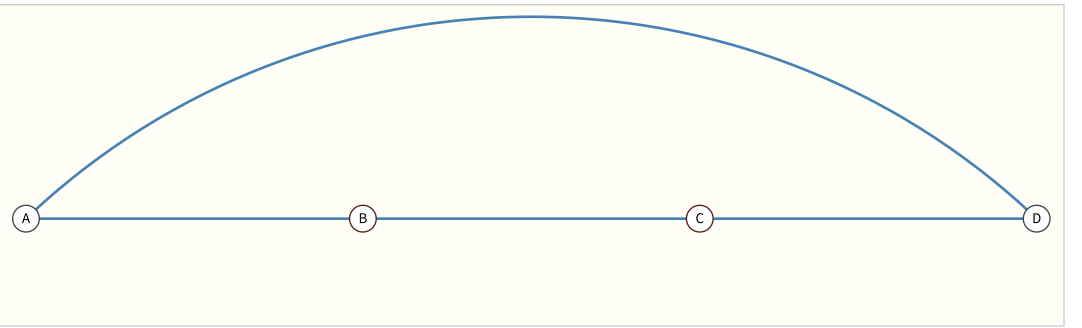
Learning per
invariant ope
area of recer
[17]

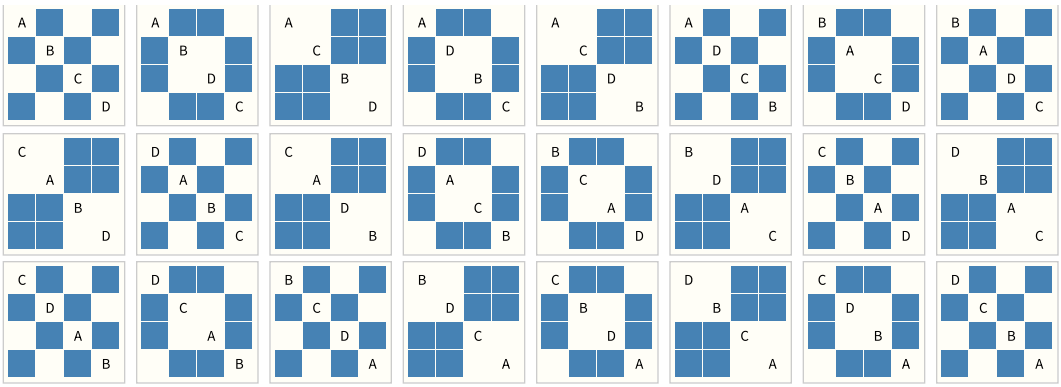
For example, the Othello graph from before can be described equivalently with these two adjacency matrices. It can also be described with every other possible permutation of the nodes.



Two adjacency matrices representing the same graph.

The example below shows every adjacency matrix that can describe this small graph of 4 nodes. This is already a significant number of adjacency matrices–for larger examples like Othello, the number is untenable.



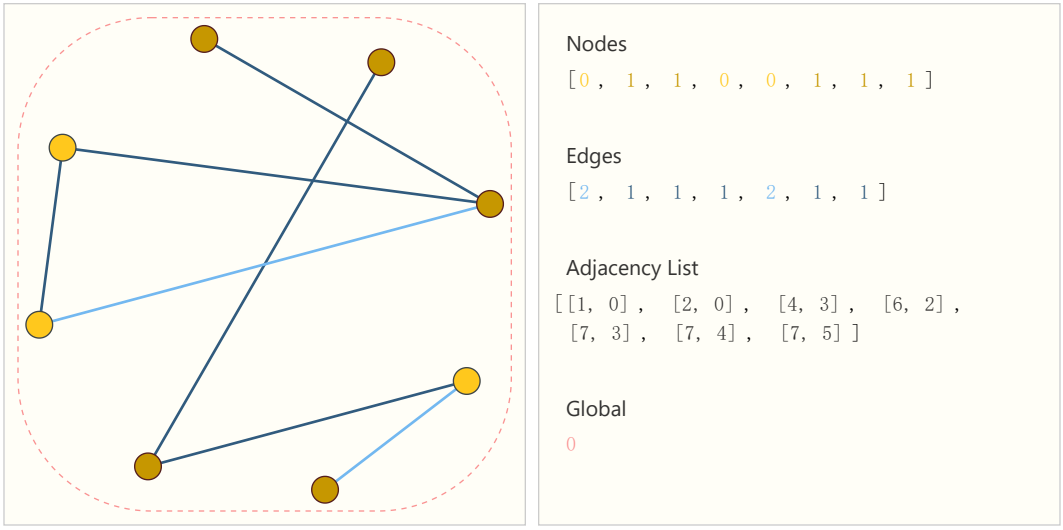


All of these adjacency matrices represent the same graph. Click on an edge to remove it on a "virtual edge" to add it and the matrices will update accordingly.

One elegant and memory-efficient way of representing sparse matrices is as adjacency lists. These describe the connectivity of edge e_k between nodes n_i and n_j as a tuple (i,j) in the k-th entry of an adjacency list. Since we expect the number of edges to be much lower than the number of entries for an adjacency matrix (n^2_{nodes}), we avoid computation and storage on the disconnected parts of the graph.

Another way is with Big-O preferable to $O(n_{edges})$, r $O(n^2_{nodes})$.

To make this notion concrete, we can see how information in different graphs might be represented under this specification:



Hover and click on the edges, nodes, and global graph marker to view and change attribute representations. On one side we have a small graph and on the other the information of the gr representation.

It should be noted that the figure uses scalar values per node/edge/global, but most practical tensor representations have vectors per graph attribute. Instead of a node tensor of size $[n_{nodes}]$ we will be dealing with node tensors of size $[n_{nodes}, node_{dim}]$. Same for the other graph attributes.

Graph Neural Networks

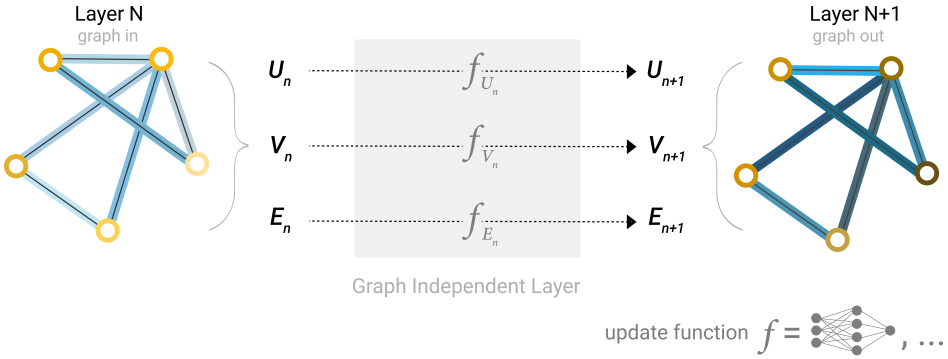
Now that the graph's description is in a matrix format that is permutation invariant, we will describe using graph neural networks (GNNs) to solve graph prediction tasks. **A GNN is an optimizable transformation on all attributes of the graph (nodes, edges, global-context) that preserves graph symmetries (permutation invariances).** We're going to build GNNs using the "message passing neural network" framework proposed by Gilmer et al.[18] using the Graph Nets architecture schematics introduced by Battaglia et al.[19] GNNs adopt a "graph-in, graph-out" architecture meaning that these model types accept a graph as input, with information loaded into its nodes,

edges and global-context, and progressively transform these embeddings, without changing the connectivity of the input graph.

The simplest GNN

With the numerical representation of graphs that we've constructed above (with vectors instead of scalars), we are now ready to build a GNN. We will start with the simplest GNN architecture, one where we learn new embeddings for all graph attributes (nodes, edges, global), but where we do not yet use the connectivity of the graph.

This GNN uses a separate multilayer perceptron (MLP) (or your favorite differentiable model) on each component of a graph; we call this a GNN layer. For each node vector, we apply the MLP and get back a learned node-vector. We do the same for each edge, learning a per-edge embedding, and also for the global-context vector, learning a single embedding for the entire graph.



A single layer of a simple GNN. A graph is the input, and each component (V,E,U) gets updated by a MLP to produce a new graph. Each function subscript indicates a separate function for a different graph attribute at the n-th layer of a GNN model.

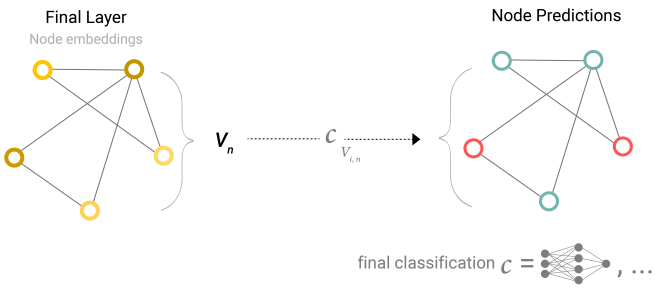
As is common with neural networks modules or layers, we can stack these GNN layers together.

Because a GNN does not update the connectivity of the input graph, we can describe the output graph of a GNN with the same adjacency list and the same number of feature vectors as the input graph. But, the output graph has updated embeddings, since the GNN has updated each of the node, edge and global-context representations.

GNN Predictions by Pooling Information

We have built a simple GNN, but how do we make predictions in any of the tasks we described above?

We will consider the case of binary classification, but this framework can easily be extended to the multi-class or regression case. If the task is to make binary predictions on nodes, and the graph already contains node information, the approach is straightforward — for each node embedding, apply a linear classifier.



For simplicity diagrams use represent gra in practice fe or embeddin more useful.

You could als block. Becau multiple ope (like a ResNe

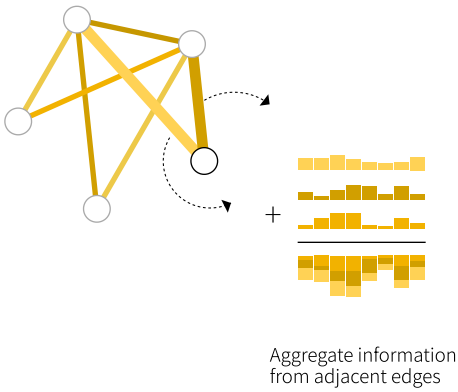
We could im network, whe anonymize u (nodes) by n and only usi data (edges) of such a sce node task we the Node-lev subsection. I club example just using th meetings be determine th Mr. Hi or Joh

However, it is not always so simple. For instance, you might have information in the graph stored in edges, but no information in nodes, but still need to make predictions on nodes. We need a way to collect information from edges and give them to nodes for prediction. We can do this by *pooling*. Pooling proceeds in two steps:

1. For each item to be pooled, *gather* each of their embeddings and concatenate them into a matrix.
2. The gathered embeddings are then *aggregated*, usually via a sum operation.

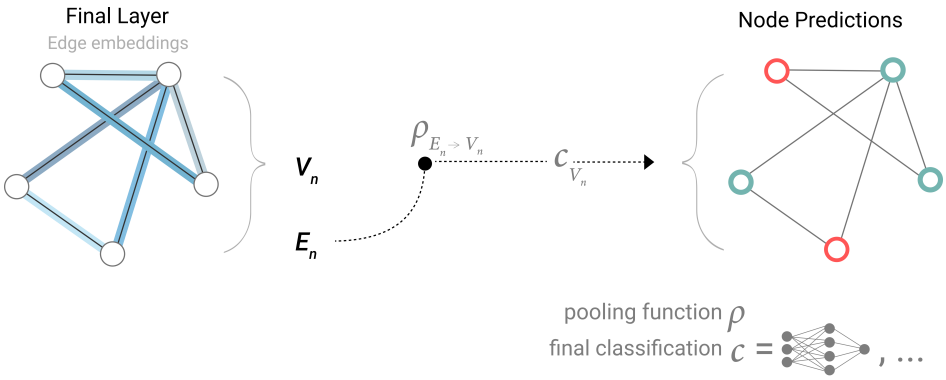
For a more in
discussion on
operations g
Comparing a
operations s

We represent the *pooling* operation by the letter ρ, and denote that we are gathering information from edges to nodes as $p_{E_n \to V_n}$.

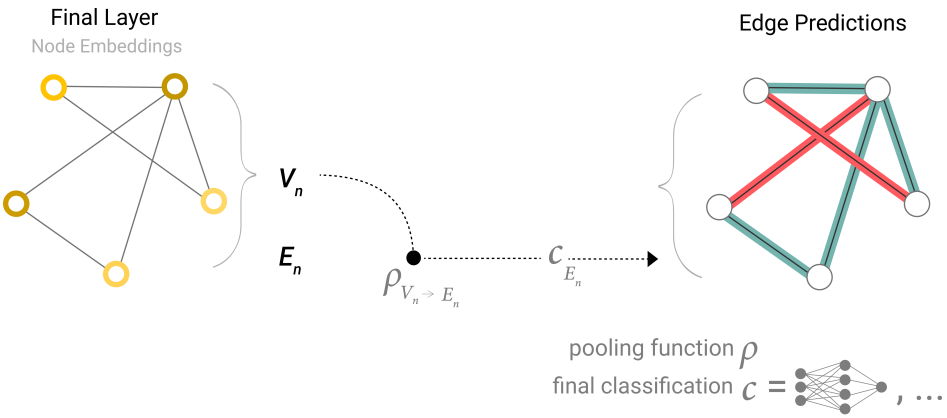


Hover over a node (black node) to visualize which edges are gathered and aggregated to produce an embedding for that target node.

So If we only have edge-level features, and are trying to predict binary node information, we can use pooling to route (or pass) information to where it needs to go. The model looks like this.

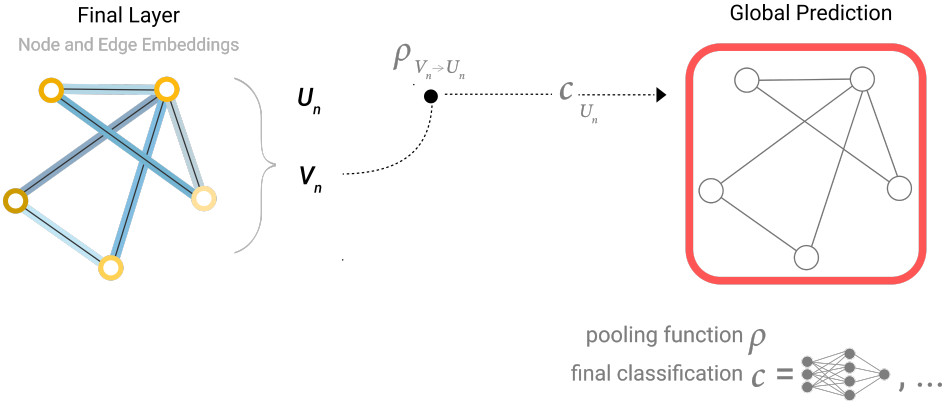


If we only have node-level features, and are trying to predict binary edge-level information, the model looks like this.



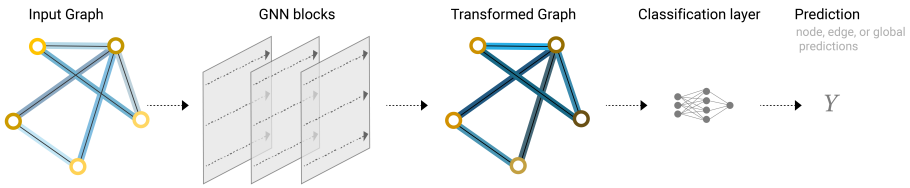
One example scenario is th specified in E sub section. recognized a entities, and predict if the relationship

If we only have node-level features, and need to predict a binary global property, we need to gather all available node information together and aggregate them. This is similar to *Global Average Pooling* layers in CNNs. The same can be done for edges.



This is a com for predictin properties. F have atomic connectivity like to know molecule (to if it has a par (rose/not ros

In our examples, the classification model c can easily be replaced with any differentiable model, or adapted to multi-class classification using a generalized linear model.



An end-to-end prediction task with a GNN model.

Now we've demonstrated that we can build a simple GNN model, and make binary predictions by routing information between different parts of the graph. This pooling technique will serve as a building block for constructing more sophisticated GNN models. If we have new graph attributes, we just have to define how to pass information from one attribute to another.

Note that in this simplest GNN formulation, we're not using the connectivity of the graph at all inside the GNN layer. Each node is processed independently, as is each edge, as well as the global context. We only use connectivity when pooling information for prediction.

Passing messages between parts of the graph

We could make more sophisticated predictions by using pooling within the GNN layer, in order to

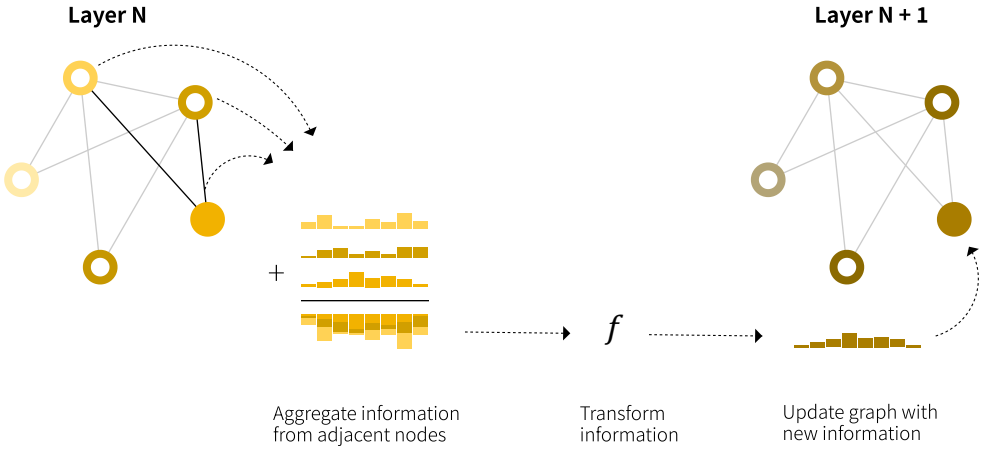
make our learned embeddings aware of graph connectivity. We can do this using *message passing* [18], where neighboring nodes or edges exchange information and influence each other's updated embeddings.
Message passing works in three steps:

1. For each node in the graph, *gather* all the neighboring node embeddings (or messages), which is the g function described above.
2. Aggregate all messages via an aggregate function (like sum).
3. All pooled messages are passed through an *update function*, usually a learned neural network.

You could als
messages, 3)
and 2) aggre
still have a p
invariant ope

Just as pooling can be applied to either nodes or edges, message passing can occur between either nodes or edges.

These steps are key for leveraging the connectivity of graphs. We will build more elaborate variants of message passing in GNN layers that yield GNN models of increasing expressiveness and power.



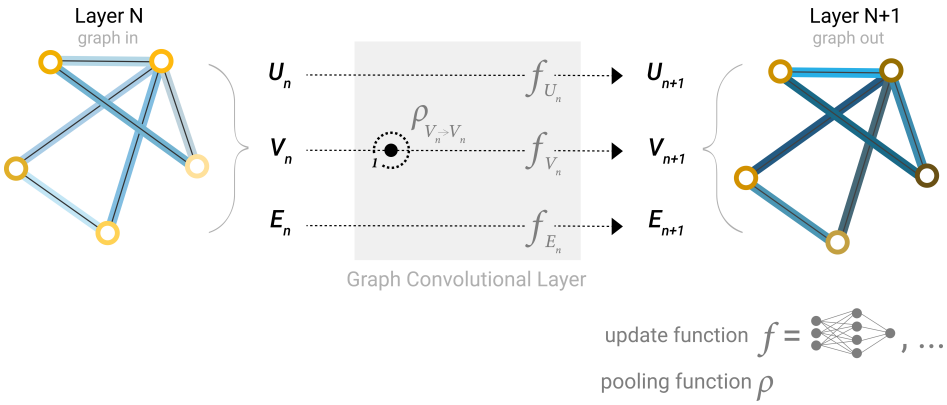
Hover over a node, to highlight adjacent nodes and visualize the adjacent embedding that would be pooled, updated and stored.

This sequence of operations, when applied once, is the simplest type of message-passing GNN layer.

This is reminiscent of standard convolution: in essence, message passing and convolution are operations to aggregate and process the information of an element's neighbors in order to update the element's value. In graphs, the element is a node, and in images, the element is a pixel. However, the number of neighboring nodes in a graph can be variable, unlike in an image where each pixel has a set number of neighboring elements.

By stacking message passing GNN layers together, a node can eventually incorporate information from across the entire graph: after three layers, a node has information about the nodes three steps away from it.

We can update our architecture diagram to include this new source of information for nodes:



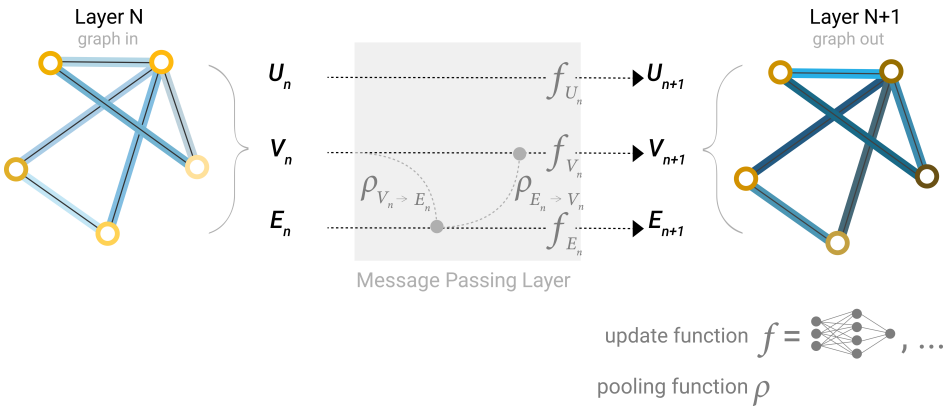
Schematic for a GCN architecture, which updates node representations of a graph by pooling neighboring nodes at a distance of one degree.

Learning edge representations

Our dataset does not always contain all types of information (node, edge, and global context). When we want to make a prediction on nodes, but our dataset only has edge information, we showed above how to use pooling to route information from edges to nodes, but only at the final prediction step of the model. We can share information between nodes and edges within the GNN layer using message passing.

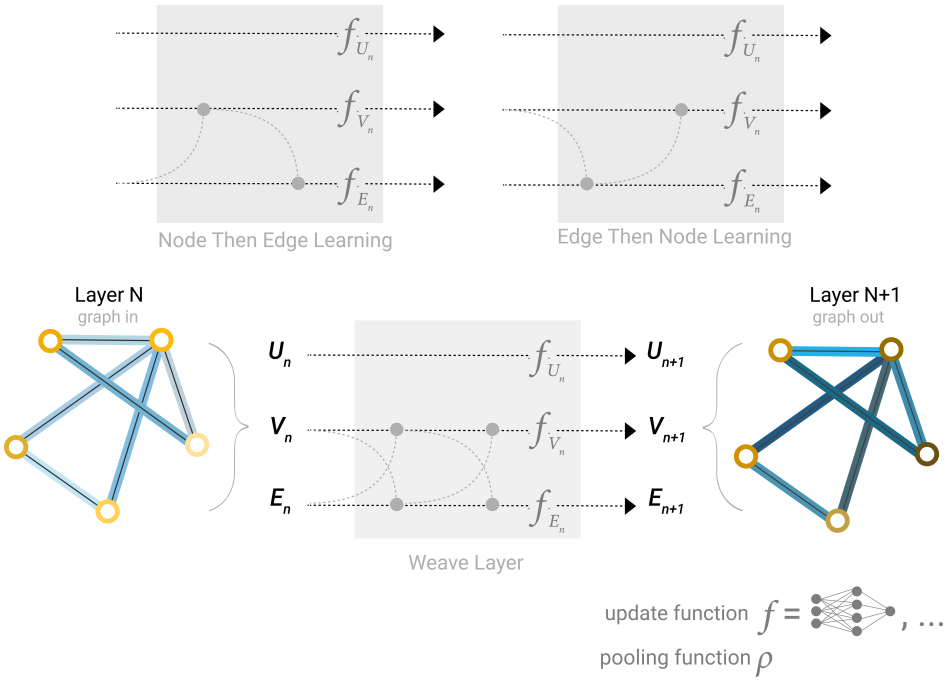
We can incorporate the information from neighboring edges in the same way we used neighboring node information earlier, by first pooling the edge information, transforming it with an update function, and storing it.

However, the node and edge information stored in a graph are not necessarily the same size or shape, so it is not immediately clear how to combine them. One way is to learn a linear mapping from the space of edges to the space of nodes, and vice versa. Alternatively, one may concatenate them together before the update function.



Architecture schematic for Message Passing layer. The first step "prepares" a message composed of information from an edge and it's connected nodes and then "passes" the message to the node.

Which graph attributes we update and in which order we update them is one design decision when constructing GNNs. We could choose whether to update node embeddings before edge embeddings, or the other way around. This is an open area of research with a variety of solutions–for example we could update in a 'weave' fashion [21] where we have four updated representations that get combined into new node and edge representations: node to node (linear), edge to edge (linear), node to edge (edge layer), edge to node (node layer).

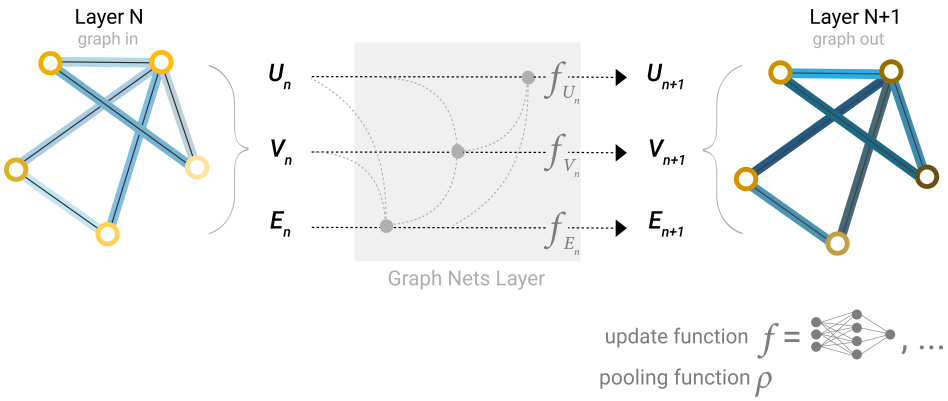


Some of the different ways we might combine edge and node representation in a GNN layer.

Adding global representations

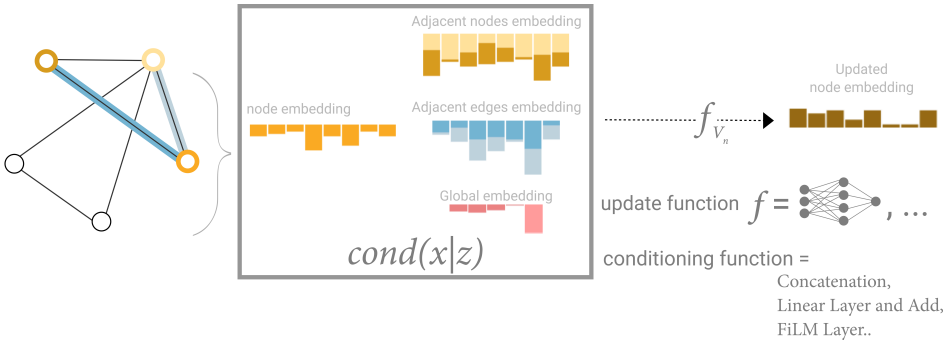
There is one flaw with the networks we have described so far: nodes that are far away from each other in the graph may never be able to efficiently transfer information to one another, even if we apply message passing several times. For one node, If we have k-layers, information will propagate at most k-steps away. This can be a problem for situations where the prediction task depends on nodes, or groups of nodes, that are far apart. One solution would be to have all nodes be able to pass information to each other. Unfortunately for large graphs, this quickly becomes computationally expensive (although this approach, called 'virtual edges', has been used for small graphs such as molecules).[18]

One solution to this problem is by using the global representation of a graph (U) which is sometimes called a **master node** [19] [18] or context vector. This global context vector is connected to all other nodes and edges in the network, and can act as a bridge between them to pass information, building up a representation for the graph as a whole. This creates a richer and more complex representation of the graph than could have otherwise been learned.



Schematic of a Graph Nets architecture leveraging global representations.

In this view all graph attributes have learned representations, so we can leverage them during pooling by conditioning the information of our attribute of interest with respect to the rest. For example, for one node we can consider information from neighboring nodes, connected edges and the global information. To condition the new node embedding on all these possible sources of information, we can simply concatenate them. Additionally we may also map them to the same space via a linear map and add them or apply a feature-wise modulation layer [22], which can be considered a type of featurize-wise attention mechanism.



Schematic for conditioning the information of one node based on three other embeddings (adjacent nodes, adjacent edges, global). This step corresponds to the node operations in the Graph Nets Layer.

GNN playground

We've described a wide range of GNN components here, but how do they actually differ in practice? This GNN playground allows you to see how these different components and architectures contribute to a GNN's ability to learn a real task.

Our playground shows a graph-level prediction task with small molecular graphs. We use the the Leffingwell Odor Dataset [23] [24], which is composed of molecules with associated odor percepts (labels). Predicting the relation of a molecular structure (graph) to its smell is a 100 year-old problem straddling chemistry, physics, neuroscience, and machine learning.

To simplify the problem, we consider only a single binary label per molecule, classifying if a molecular graph smells "pungent" or not, as labeled by a professional perfumer. We say a molecule has a "pungent" scent if it has a strong, striking smell. For example, garlic and mustard, which might contain the molecule *allyl alcohol* have this quality. The molecule *piperitone*, often used for peppermint-flavored candy, is also described as having a pungent smell.

We represent each molecule as a graph, where atoms are nodes containing a one-hot encoding for its atomic identity (Carbon, Nitrogen, Oxygen, Fluorine) and bonds are edges containing a one-hot encoding its bond type (single, double, triple or aromatic).

Our general modeling template for this problem will be built up using sequential GNN layers, followed by a linear model with a sigmoid activation for classification. The design space for our GNN has many levers that can customize the model:

1. The number of GNN layers, also called the *depth*.
2. The dimensionality of each attribute when updated. The update function is a 1-layer MLP with a relu activation function and a layer norm for normalization of activations.
3. The aggregation function used in pooling: max, mean or sum.

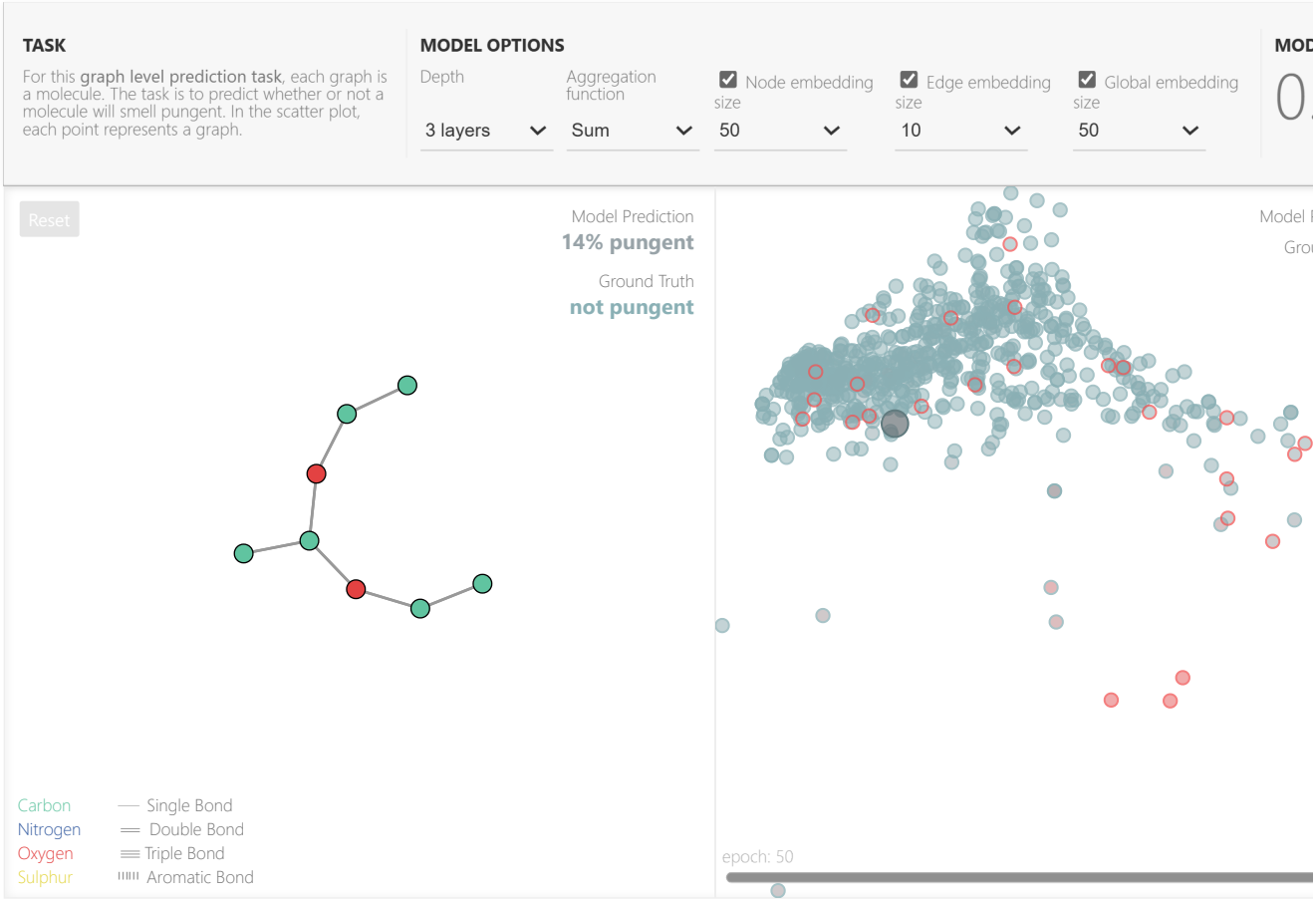
4. The graph attributes that get updated, or styles of message passing: nodes, edges and global representation. We control these via boolean toggles (on or off). A baseline model would be a graph-independent GNN (all message-passing off) which aggregates all data at the end into a single global attribute. Toggling on all message-passing functions yields a GraphNets architecture.

To better understand how a GNN is learning a task-optimized representation of a graph, we also look at the penultimate layer activations of the GNN. These 'graph embeddings' are the outputs of the GNN model right before prediction. Since we are using a generalized linear model for prediction, a linear mapping is enough to allow us to see how we are learning representations around the decision boundary.

Since these are high dimensional vectors, we reduce them to 2D via principal component analysis (PCA). A perfect model would visibility separate labeled data, but since we are reducing dimensionality and also have imperfect models, this boundary might be harder to see.

Play around with different model architectures to build your intuition. For example, see if you can edit the molecule on the left to make the model prediction increase. Do the same edits have the same effects for different model architectures?

This playgrou
live on the b



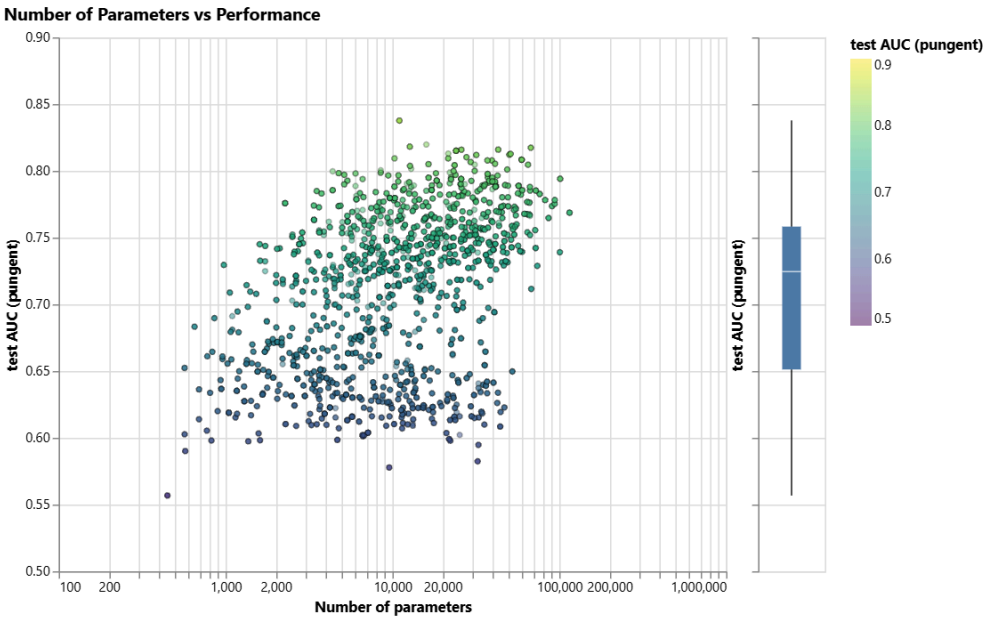
Edit the molecule to see how the prediction changes, or change the model params to load a different model. Select a different molecule in the scatter plot.

Some empirical GNN design lessons

When exploring the architecture choices above, you might have found some models have better performance than others. Are there some clear GNN design choices that will give us better performance? For example, do deeper GNN models perform better than shallower ones? or is there a clear choice between aggregation functions? The answers are going to depend on the data, [25] [26], and even different ways of featurizing and constructing graphs can give different answers.

With the following interactive figure, we explore the space of GNN architectures and the performance of this task across a few major design choices: Style of message passing, the dimensionality of embeddings, number of layers, and aggregation operation type.

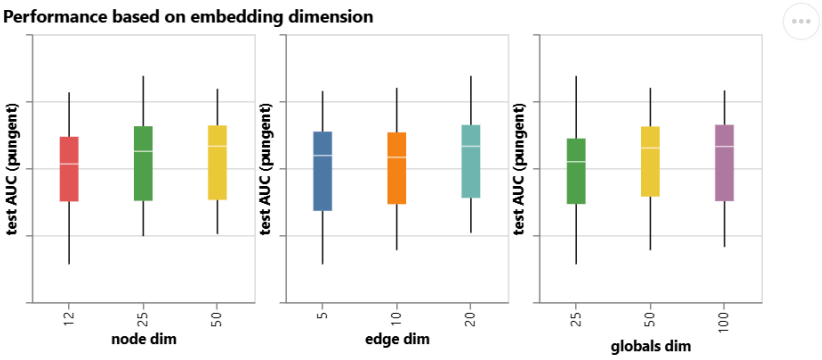
Each point in the scatter plot represents a model: the x axis is the number of trainable variables, and the y axis is the performance. Hover over a point to see the GNN architecture parameters.



Scatterplot of each model's performance vs its number of trainable variables. Hover over a point to see the GNN architecture parameters.

The first thing to notice is that, surprisingly, a higher number of parameters does correlate with higher performance. GNNs are a very parameter-efficient model type: for even a small number of parameters (3k) we can already find models with high performance.

Next, we can look at the distributions of performance aggregated based on the dimensionality of the learned representations for different graph attributes.



Aggregate performance of models across varying node, edge, and global dimensions.

We can notice that models with higher dimensionality tend to have better mean and lower bound performance but the same trend is not found for the maximum. Some of the top-performing models can be found for smaller dimensions. Since higher dimensionality is going to also involve a higher number of parameters, these observations go in hand with the previous figure.

Next we can see the breakdown of performance based on the number of GNN layers.

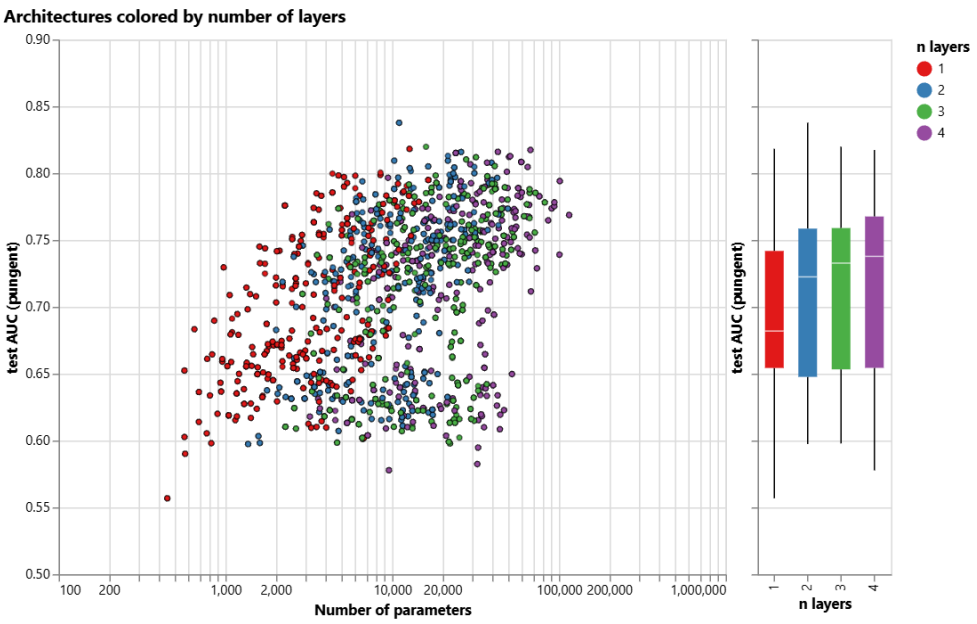


Chart of number of layers vs model performance, and scatterplot of model performance vs number of parameters. Each point is colored by the number of layers. Hover over a point to see the GNN architecture parameters.

The box plot shows a similar trend, while the mean performance tends to increase with the number of layers, the best performing models do not have three or four layers, but two. Furthermore, the lower bound for performance decreases with four layers. This effect has been observed before, GNN with a higher number of layers will broadcast information at a higher distance and can risk having their node representations 'diluted' from many successive iterations [27].

Does our dataset have a preferred aggregation operation? Our following figure breaks down performance in terms of aggregation type.

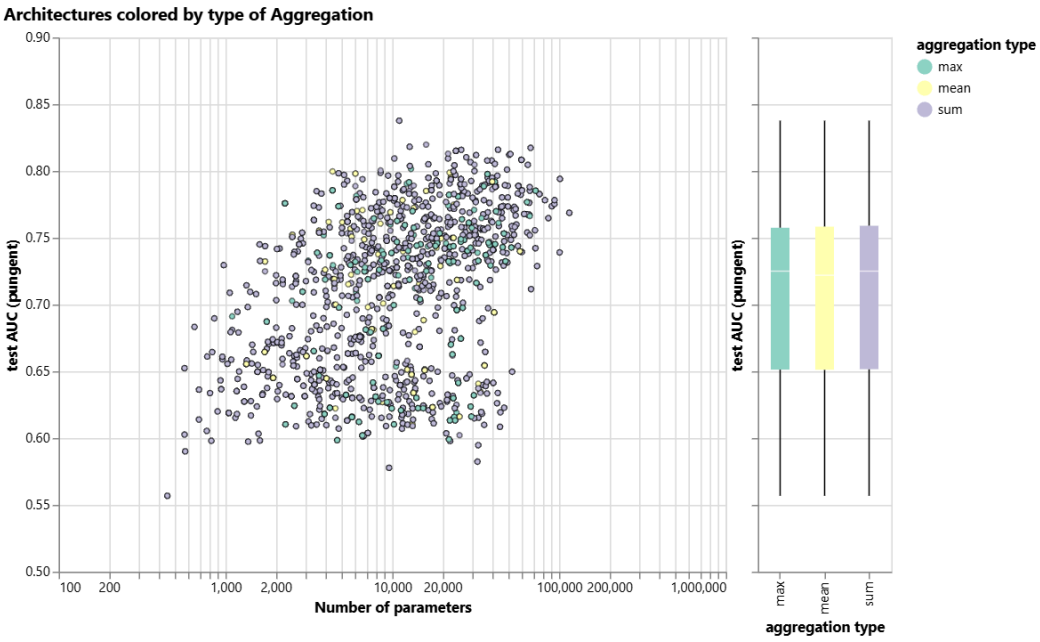


Chart of aggregation type vs model performance, and scatterplot of model performance vs number of parameters. Each point is colored by aggregation type. Hover over a point to see the GNN architecture parameters.

Overall it appears that sum has a very slight improvement on the mean performance, but max or mean can give equally good models. This is useful to contextualize when looking at the discriminatory/expressive capabilities of aggregation operations .

The previous explorations have given mixed messages. We can find mean trends where more complexity gives better performance but we can find clear counterexamples where models with fewer parameters, number of layers, or dimensionality perform better. One trend that is much clearer is about the number of attributes that are passing information to each other.

Here we break down performance based on the style of message passing. On both extremes, we consider models that do not communicate between graph entities ("none") and models that have messaging passed between nodes, edges, and globals.

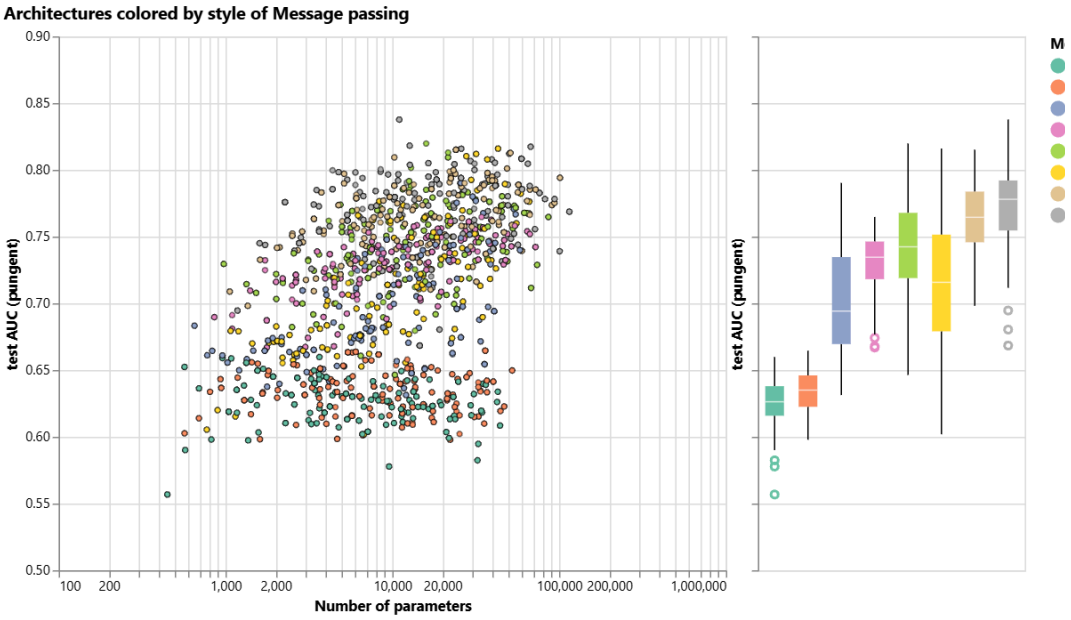


Chart of message passing vs model performance, and scatterplot of model performance vs number of parameters. Each point is colored by message passing. Hover over a point to see the GNN architecture parameters

Overall we see that the more graph attributes are communicating, the better the performance of the average model. Our task is centered on global representations, so explicitly learning this attribute also tends to improve performance. Our node representations also seem to be more useful than edge representations, which makes sense since more information is loaded in these attributes.

There are many directions you could go from here to get better performance. We wish two highlight two general directions, one related to more sophisticated graph algorithms and another towards the graph itself.

Up until now, our GNN is based on a neighborhood-based pooling operation. There are some graph concepts that are harder to express in this way, for example a linear graph path (a connected chain of nodes). Designing new mechanisms in which graph information can be extracted, executed and propagated in a GNN is one current research area [28], [29], [30], [31].

One of the frontiers of GNN research is not making new models and architectures, but "how to construct graphs", to be more precise, imbuing graphs with additional structure or relations that can be leveraged. As we loosely saw, the more graph attributes are communicating the more we tend to have better models. In this particular case, we could consider making molecular graphs more feature rich, by adding additional spatial relationships between nodes, adding edges that are not bonds, or explicit learnable relationships between subgraphs.

See more in graphs.

Into the Weeds

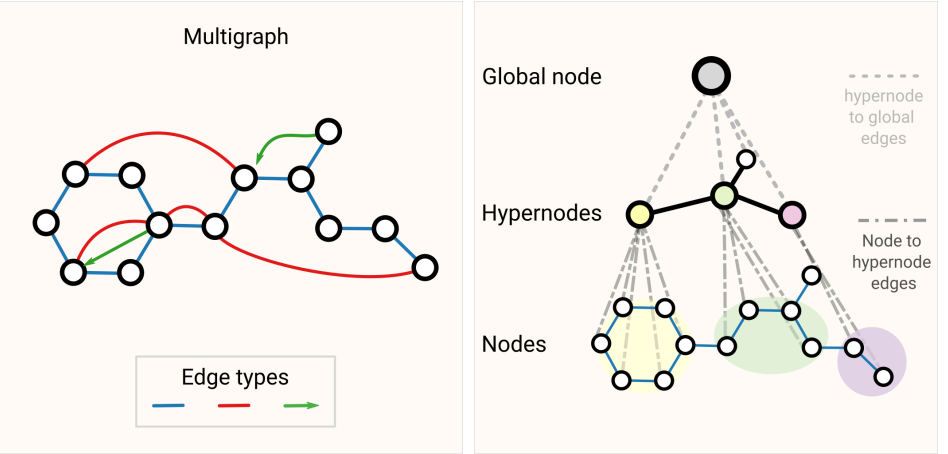
Next, we have a few sections on a myriad of graph-related topics that are relevant for GNNs.

Other types of graphs (multigraphs, hypergraphs, hypernodes, hierarchical graphs)

While we only described graphs with vectorized information for each attribute, graph structures are more flexible and can accommodate other types of information. Fortunately, the message passing framework is flexible enough that often adapting GNNs to more complex graph structures is about defining how information is passed and updated by new graph attributes.

For example, we can consider multi-edge graphs or *multigraphs* [32], where a pair of nodes can share multiple types of edges, this happens when we want to model the interactions between nodes differently based on their type. For example with a social network, we can specify edge types based on the type of relationships (acquaintance, friend, family). A GNN can be adapted by having different types of message passing steps for each edge type. We can also consider nested graphs, where for example a node represents a graph, also called a hypernode graph. [33] Nested graphs are useful for representing hierarchical information. For example, we can consider a network of molecules, where a node represents a molecule and an edge is shared between two molecules if we have a way (reaction) of transforming one to the other [34] [35]. In this case, we can learn on a nested graph by having a GNN that learns representations at the molecule level and another at the reaction network level, and alternate between them during training.

Another type of graph is a hypergraph [36], where an edge can be connected to multiple nodes instead of just two. For a given graph, we can build a hypergraph by identifying communities of nodes and assigning a hyper-edge that is connected to all nodes in a community.



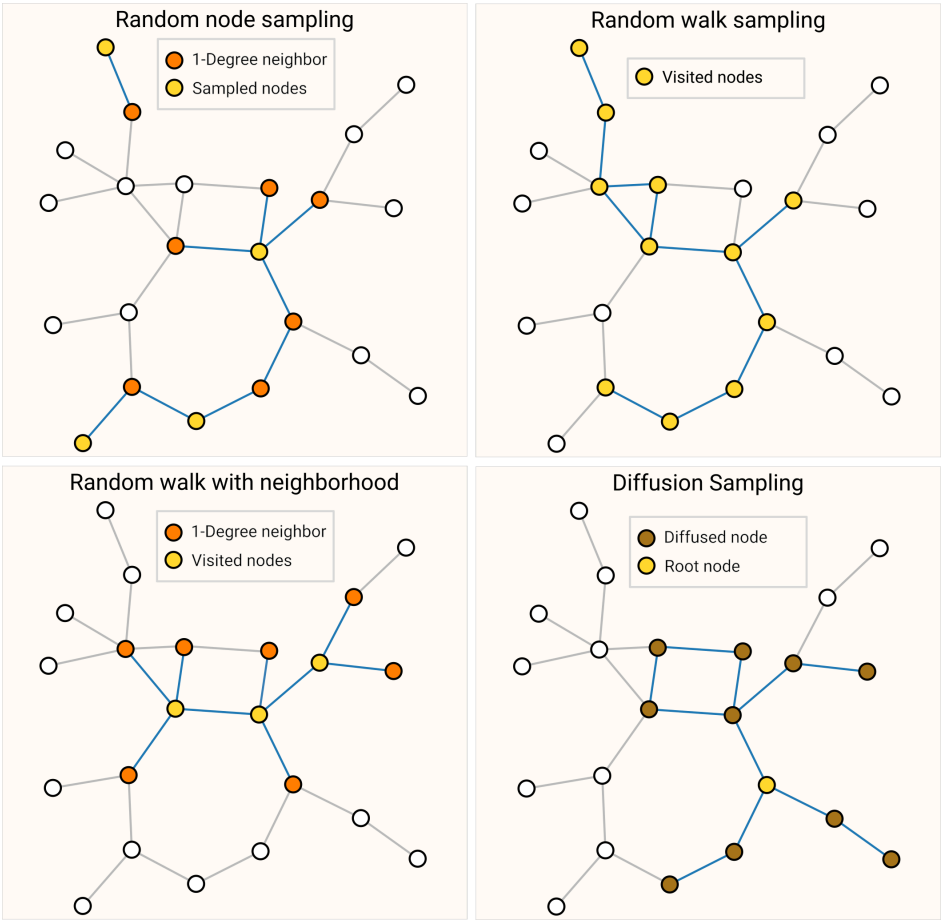
Schematic of more complex graphs. On the left we have an example of a multigraph with three edge types, including a directed edge. On the right we have a three-level hierarchical graph, the intermediate level nodes are hypernodes.

How to train and design GNNs that have multiple types of graph attributes is a current area of research [37], [38].

Sampling Graphs and Batching in GNNs

A common practice for training neural networks is to update network parameters with gradients calculated on randomized constant size (batch size) subsets of the training data (mini-batches). This practice presents a challenge for graphs due to the variability in the number of nodes and edges adjacent to each other, meaning that we cannot have a constant batch size. The main idea for batching with graphs is to create subgraphs that preserve essential properties of the larger graph. This graph sampling operation is highly dependent on context and involves sub-selecting nodes and edges from a graph. These operations might make sense in some contexts (citation networks) and in others, these might be too strong of an operation (molecules, where a subgraph simply represents a new, smaller molecule). How to sample a graph is an open research question. [39] If we care about preserving structure at a neighborhood level, one way would be to randomly sample a uniform number of nodes, our *node-set*. Then add neighboring nodes of distance k adjacent to the

node-set, including their edges.[40] Each neighborhood can be considered an individual graph and a GNN can be trained on batches of these subgraphs. The loss can be masked to only consider the node-set since all neighboring nodes would have incomplete neighborhoods. A more efficient strategy might be to first randomly sample a single node, expand its neighborhood to distance k, and then pick the other node within the expanded set. These operations can be terminated once a certain amount of nodes, edges, or subgraphs are constructed. If the context allows, we can build constant size neighborhoods by picking an initial node-set and then sub-sampling a constant number of nodes (e.g randomly, or via a random walk or Metropolis algorithm[41]).



Four different ways of sampling the same graph. Choice of sampling strategy depends highly on context since they will generate different distributions of graph statistics (# nodes, #edges, etc.). For highly connected graphs, edges can be also subsampled.

Sampling a graph is particularly relevant when a graph is large enough that it cannot be fit in memory. Inspiring new architectures and training strategies such as Cluster-GCN [42] and GraphSaint [43]. We expect graph datasets to continue growing in size in the future.

Inductive biases

When building a model to solve a problem on a specific kind of data, we want to specialize our models to leverage the characteristics of that data. When this is done successfully, we often see better predictive performance, lower training time, fewer parameters and better generalization.

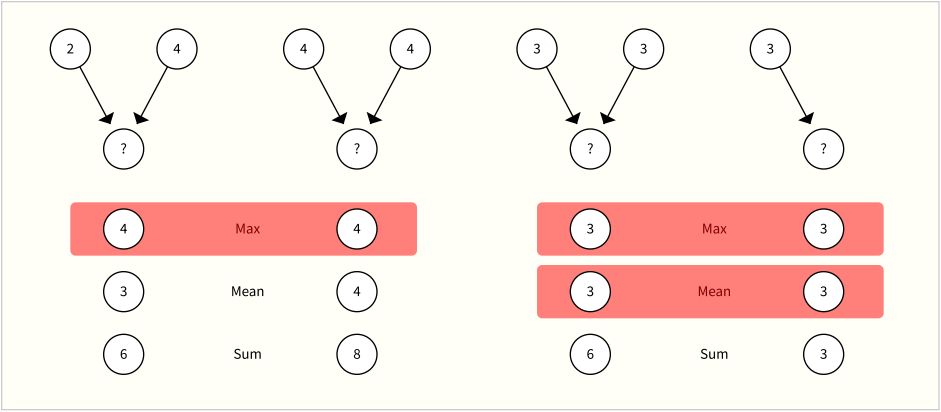
When labeling on images, for example, we want to take advantage of the fact that a dog is still a dog whether it is in the top-left or bottom-right corner of an image. Thus, most image models use convolutions, which are translation invariant. For text, the order of the tokens is highly important, so recurrent neural networks process data sequentially. Further, the presence of one token (e.g. the word 'not') can affect the meaning of the rest of a sentence, and so we need components that can 'attend' to other parts of the text, which transformer models like BERT and GPT-3 can do. These are some examples of inductive biases, where we are identifying symmetries or regularities in the data and adding modelling components that take advantage of these properties.

In the case of graphs, we care about how each graph component (edge, node, global) is related to each other so we seek models that have a relational inductive bias. [19] A model should preserve explicit relationships between entities (adjacency matrix) and preserve graph symmetries (permutation invariance). We expect problems where the interaction between entities is important will benefit from a graph structure. Concretely, this means designing transformation on sets: the order of operation on nodes or edges should not matter and the operation should work on a variable number of inputs.

Comparing aggregation operations

Pooling information from neighboring nodes and edges is a critical step in any reasonably powerful GNN architecture. Because each node has a variable number of neighbors, and because we want a differentiable method of aggregating this information, we want to use a smooth aggregation operation that is invariant to node ordering and the number of nodes provided.

Selecting and designing optimal aggregation operations is an open research topic. [44] A desirable property of an aggregation operation is that similar inputs provide similar aggregated outputs, and vice-versa. Some very simple candidate permutation-invariant operations are sum, mean, and max. Summary statistics like variance also work. All of these take a variable number of inputs, and provide an output that is the same, no matter the input ordering. Let's explore the difference between these operations.



No pooling type can always distinguish between graph pairs such as max pooling on the left and sum / mean pooling on the right.

There is no operation that is uniformly the best choice. The mean operation can be useful when nodes have a highly-variable number of neighbors or you need a normalized view of the features of a local neighborhood. The max operation can be useful when you want to highlight single salient features in local neighborhoods. Sum provides a balance between these two, by providing a snapshot of the local distribution of features, but because it is not normalized, can also highlight outliers. In practice, sum is commonly used.

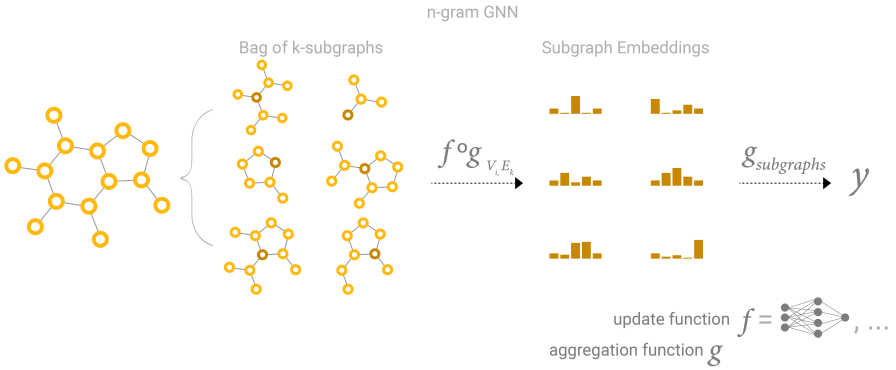
Designing aggregation operations is an open research problem that intersects with machine learning on sets.[45] New approaches such as Principal Neighborhood aggregation[27] take into account several aggregation operations by concatenating them and adding a scaling function that depends on the degree of connectivity of the entity to aggregate. Meanwhile, domain specific aggregation operations can also be designed. One example lies with the "Tetrahedral Chirality" aggregation operators [46].

GCN as subgraph function approximators

Another way to see GCN (and MPNN) of k-layers with a 1-degree neighbor lookup is as a neural network that operates on learned embeddings of subgraphs of size k.[47] [44]

When focusing on one node, after k-layers, the updated node representation has a limited viewpoint of all neighbors up to k-distance, essentially a subgraph representation. Same is true for edge representations.

So a GCN is collecting all possible subgraphs of size k and learning vector representations from the vantage point of one node or edge. The number of possible subgraphs can grow combinatorially, so enumerating these subgraphs from the beginning vs building them dynamically as in a GCN, might be prohibitive.



Edges and the Graph Dual

One thing to note is that edge predictions and node predictions, while seemingly different, often reduce to the same problem: an edge prediction task on a graph G can be phrased as a node-level prediction on G's dual.

To obtain G's dual, we can convert nodes to edges (and edges to nodes). A graph and its dual contain the same information, just expressed in a different way. Sometimes this property makes solving problems easier in one representation than another, like frequencies in Fourier space. In short, to solve an edge classification problem on G, we can think about doing graph convolutions on G's dual (which is the same as learning edge representations on G), this idea was developed with Dual-Primal Graph Convolutional Networks.[48]

Graph convolutions as matrix multiplications, and matrix multiplications as walks on a graph

We've talked a lot about graph convolutions and message passing, and of course, this raises the question of how do we implement these operations in practice? For this section, we explore some of the properties of matrix multiplication, message passing, and its connection to traversing a graph.

The first point we want to illustrate is that the matrix multiplication of an adjacent matrix A $n_{nodes} \times n_{nodes}$ with a node feature matrix X of size $n_{nodes} \times node_{dim}$ implements an simple message passing with a summation aggregation. Let the matrix be $B = AX$, we can observe that any entry B_{ij} can be expressed as $< A_{row_i} \dot{X}_{column_j} >= A_{i,1}X_{1,j} + A_{i,2}X_{2,j} + \ldots + A_{i,n}X_{n,j} = \sum_{A_{i,k}>0} X_{k,j}$. Because $A_{i,k}$ are binary entries only when a edge exists between $node_i$ and $node_k$, the inner product is essentially "gathering" all node features values of dimension j" that share an edge with $node_i$. It should be noted that this message passing is not updating the representation of the node features, just pooling neighboring node features. But this can be easily adapted by passing X through your favorite differentiable transformation (e.g. MLP) before or after the matrix multiply.

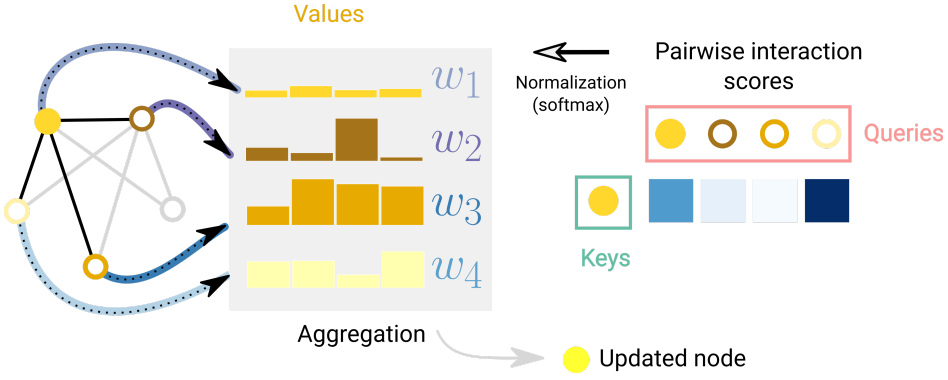
From this view, we can appreciate the benefit of using adjacency lists. Due to the expected sparsity of A we don't have to sum all values where $A_{i,j}$ is zero. As long as we have an operation to gather values based on an index, we should be able to just retrieve positive entries. Additionally, this matrix multiply-free approach frees us from using summation as an aggregation operation.

We can imagine that applying this operation multiple times allows us to propagate information at greater distances. In this sense, matrix multiplication is a form of traversing over a graph. This relationship is also apparent when we look at powers A^K of the adjacency matrix. If we consider the matrix A^2, the term A^2_{ij} counts all walks of length 2 from $node_i$ to $node_j$ and can be expressed as the inner product $< A_{row_i}, A_{column_j} >= A_{i,1}A_{1,j} + A_{i,2}A_{2,j} + \ldots + A_{i,n}An, j$. The intuition is that the first term $a_{i,1}\,a_{1,j}$ is only positive under two conditions, there is edge that connects $node_i$ to $node_1$ and another edge that connects $node_1$ to $node_j$. In other words, both edges form a path of length 2 that goes from $node_i$ to $node_j$ passing by $node_1$. Due to the summation, we are counting over all possible intermediate nodes. This intuition carries over when we consider $A^3 = A\,A^2$.. and so on to A^k.

There are deeper connections on how we can view matrices as graphs to explore [49] [50] [51].

Graph Attention Networks

Another way of communicating information between graph attributes is via attention. [52] For example, when we consider the sum-aggregation of a node and its 1-degree neighboring nodes we could also consider using a weighted sum.The challenge then is to associate weights in a permutation invariant fashion. One approach is to consider a scalar scoring function that assigns weights based on pairs of nodes ($f(node_i, node_j)$). In this case, the scoring function can be interpreted as a function that measures how relevant a neighboring node is in relation to the center node. Weights can be normalized, for example with a softmax function to focus most of the weight on a neighbor most relevant for a node in relation to a task. This concept is the basis of Graph Attention Networks (GAT) [53] and Set Transformers [54]. Permutation invariance is preserved, because scoring works on pairs of nodes. A common scoring function is the inner product and nodes are often transformed before scoring into query and key vectors via a linear map to increase the expressivity of the scoring mechanism. Additionally for interpretability, the scoring weights can be used as a measure of the importance of an edge in relation to a task.

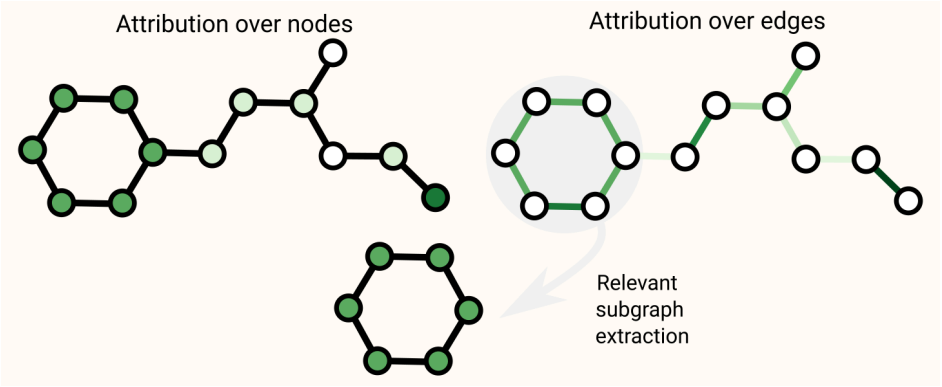


Schematic of attention over one node with respect to it's adjacent nodes. For each edge an interaction score is computed, normalized and used to weight node embeddings.

Additionally, transformers can be viewed as GNNs with an attention mechanism [55]. Under this view, the transformer models several elements (i.e. character tokens) as nodes in a fully connected graph and the attention mechanism is assigning edge embeddings to each node-pair which are used to compute attention weights. The difference lies in the assumed pattern of connectivity between entities, a GNN is assuming a sparse pattern and the Transformer is modelling all connections.

Graph explanations and attributions

When deploying GNN in the wild we might care about model interpretability for building credibility, debugging or scientific discovery. The graph concepts that we care to explain vary from context to context. For example, with molecules we might care about the presence or absence of particular subgraphs [56], while in a citation network we might care about the degree of connectedness of an article. Due to the variety of graph concepts, there are many ways to build explanations. GNNExplainer [57] casts this problem as extracting the most relevant subgraph that is important for a task. Attribution techniques [58] assign ranked importance values to parts of a graph that are relevant for a task. Because realistic and challenging graph problems can be generated synthetically, GNNs can serve as a rigorous and repeatable testbed for evaluating attribution techniques [59].



Schematic of some explanability techniques on graphs. Attributions assign ranked values to graph attributes. Rankings can be used as a basis to extract connected subgraphs that might be relevant to a task.

Generative modelling

Besides learning predictive models on graphs, we might also care about learning a generative model for graphs. With a generative model we can generate new graphs by sampling from a learned distribution or by completing a graph given a starting point. A relevant application is in the design of new drugs, where novel molecular graphs with specific properties are desired as candidates to treat a disease.

A key challenge with graph generative models lies in modelling the topology of a graph, which can vary dramatically in size and has N_{nodes}^2 terms. One solution lies in modelling the adjacency matrix directly like an image with an autoencoder framework. [60] The prediction of the presence or absence of an edge is treated as a binary classification task. The N_{nodes}^2 term can be avoided by only predicting known edges and a subset of the edges that are not present. The graphVAE learns to model positive patterns of connectivity and some patterns of non-connectivity in the adjacency matrix.

Another approach is to build a graph sequentially, by starting with a graph and applying discrete actions such as addition or subtraction of nodes and edges iteratively. To avoid estimating a gradient for discrete actions we can use a policy gradient. This has been done via an auto-regressive model, such a RNN [61], or in a reinforcement learning scenario. [62] Furthermore, sometimes graphs can be modeled as just sequences with grammar elements. [63] [64]

Final thoughts

Graphs are a powerful and rich structured data type that have strengths and challenges that are very different from those of images and text. In this article, we have outlined some of the milestones that researchers have come up with in building neural network based models that process graphs. We have walked through some of the important design choices that must be made when using these architectures, and hopefully the GNN playground can give an intuition on what the empirical results of these design choices are. The success of GNNs in recent years creates a great opportunity for a wide range of new problems, and we are excited to see what the field will bring.

Acknowledgments

We are deeply grateful to Andy Coenen, Brian Lee, Chaitanya K. Joshi, Ed Chi, Humza Iqbal, Fernanda Viegas, Jasper Snoek, Jennifer Wei, Martin Wattenberg, Patricia Robinson, Wesley Qian and Yiliu Wang for their helpful feedback and suggestions, and to Michael Terry for his code reviews.

Many of our GNN architecture diagrams are based on the Graph Nets diagram [19].

Author Contributions

All authors contributed to writing.

Adam Pearce and Emily Reif made the interactive diagrams and set up the figure aesthetics. Benjamin Sanchez-Lengeling and Emily Reif made some of the initial image sketches. Alexander B. Wiltschko provided editing and writing guidance.

Discussion and Review

Review #1 - Chaitanya K. Joshi
Review #2 - Patricia Robinson
Review #3 - Humza Iqbal

References

1. **Understanding Convolutions on Graphs**
   Daigavane, A., Ravindran, B. and Aggarwal, G., 2021. Distill. DOI: 10.23915/distill.00032

2. **The Graph Neural Network Model**
   Scarselli, F., Gori, M., Tsoi, A.C., Hagenbuchner, M. and Monfardini, G., 2009. IEEE Transactions on Neural Networks, Vol 20(1), pp. 61--80.

3. **A Deep Learning Approach to Antibiotic Discovery**
   Stokes, J.M., Yang, K., Swanson, K., Jin, W., Cubillos-Ruiz, A., Donghia, N.M., MacNair, C.R., French, S., Carfrae, L.A., Bloom-Ackermann, Z., Tran, V.M., Chiappino-Pepe, A., Badran, A.H., Andrews, I.W., Chory, E.J., Church, G.M., Brown, E.D., Jaakkola, T.S., Barzilay, R. and Collins, J.J., 2020. Cell, Vol 181(2), pp. 475--483.

4. **Learning to simulate complex physics with graph networks**
   Sanchez-Gonzalez, A., Godwin, J., Pfaff, T., Ying, R., Leskovec, J. and Battaglia, P.W., 2020.

5. **Fake News Detection on Social Media using Geometric Deep Learning**
   Monti, F., Frasca, F., Eynard, D., Mannion, D. and Bronstein, M.M., 2019.

6. **Traffic prediction with advanced Graph Neural Networks**
   *, O.L. and Perez, L..

7. **Pixie: A System for Recommending 3+ Billion Items to 200+ Million Users in {Real-Time}**
   Eksombatchai, C., Jindal, P., Liu, J.Z., Liu, Y., Sharma, R., Sugnet, C., Ulrich, M. and Leskovec, J., 2017.

8. **Convolutional Networks on Graphs for Learning Molecular Fingerprints**
   Duvenaud, D., Maclaurin, D., Aguilera-Iparraguirre, J., Gomez-Bombarelli, R., Hirzel, T., Aspuru-Guzik, A. and Adams, R.P., 2015.

9. **Distributed Representations of Words and Phrases and their Compositionality**
Mikolov, T., Sutskever, I., Chen, K., Corrado, G. and Dean, J., 2013.

10. **BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding**
Devlin, J., Chang, M., Lee, K. and Toutanova, K., 2018.

11. **Glove: Global Vectors for Word Representation**
Pennington, J., Socher, R. and Manning, C., 2014. Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP).

12. **Learning to Represent Programs with Graphs**
Allamanis, M., Brockschmidt, M. and Khademi, M., 2017.

13. **Deep Learning for Symbolic Mathematics**
Lample, G. and Charton, F., 2019.

14. **KONECT**
Kunegis, J., 2013. Proceedings of the 22nd International Conference on World Wide Web - WWW '13 Companion.

15. **An Information Flow Model for Conflict and Fission in Small Groups**
Zachary, W.W., 1977. J. Anthropol. Res., Vol 33(4), pp. 452--473. The University of Chicago Press.

16. **Learning Latent Permutations with Gumbel-Sinkhorn Networks**
Mena, G., Belanger, D., Linderman, S. and Snoek, J., 2018.

17. **Janossy Pooling: Learning Deep Permutation-Invariant Functions for Variable-Size Inputs**
Murphy, R.L., Srinivasan, B., Rao, V. and Ribeiro, B., 2018.

18. **Neural Message Passing for Quantum Chemistry**
Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O. and Dahl, G.E., 2017. Proceedings of the 34th International Conference on Machine Learning, Vol 70, pp. 1263--1272. PMLR.

19. **Relational inductive biases, deep learning, and graph networks**
Battaglia, P.W., Hamrick, J.B., Bapst, V., Sanchez-Gonzalez, A., Zambaldi, V., Malinowski, M., Tacchetti, A., Raposo, D., Santoro, A., Faulkner, R., Gulcehre, C., Song, F., Ballard, A., Gilmer, J., Dahl, G., Vaswani, A., Allen, K., Nash, C., Langston, V., Dyer, C., Heess, N., Wierstra, D., Kohli, P., Botvinick, M., Vinyals, O., Li, Y. and Pascanu, R., 2018.

20. **Deep Sets**
Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. and Smola, A., 2017.

21. **Molecular graph convolutions: moving beyond fingerprints**
Kearnes, S., McCloskey, K., Berndl, M., Pande, V. and Riley, P., 2016. J. Comput. Aided Mol. Des., Vol 30(8), pp. 595--608.

22. **Feature-wise transformations**
Dumoulin, V., Perez, E., Schucher, N., Strub, F., Vries, H.d., Courville, A. and Bengio, Y., 2018. Distill, Vol 3(7), pp. e11.

23. **Leffingwell Odor Dataset**
Sanchez-Lengeling, B., Wei, J.N., Lee, B.K., Gerkin, R.C., Aspuru-Guzik, A. and Wiltschko, A.B., 2020.

24. **Machine Learning for Scent: Learning Generalizable Perceptual Representations of Small Molecules**
Sanchez-Lengeling, B., Wei, J.N., Lee, B.K., Gerkin, R.C., Aspuru-Guzik, A. and Wiltschko, A.B., 2019.

25. **Benchmarking Graph Neural Networks**
Dwivedi, V.P., Joshi, C.K., Laurent, T., Bengio, Y. and Bresson, X., 2020.

26. **Design Space for Graph Neural Networks**
You, J., Ying, R. and Leskovec, J., 2020.

27. **Principal Neighbourhood Aggregation for Graph Nets**
Corso, G., Cavalleri, L., Beaini, D., Lio, P. and Velickovic, P., 2020.

28. **Graph Traversal with Tensor Functionals: A Meta-Algorithm for Scalable Learning**
Markowitz, E., Balasubramanian, K., Mirtaheri, M., Abu-El-Haija, S., Perozzi, B., Ver Steeg, G. and Galstyan, A., 2021.

29. **Graph Neural Tangent Kernel: Fusing Graph Neural Networks with Graph Kernels**
Du, S.S., Hou, K., Poczos, B., Salakhutdinov, R., Wang, R. and Xu, K., 2019.

30. **Representation Learning on Graphs with Jumping Knowledge Networks**
Xu, K., Li, C., Tian, Y., Sonobe, T., Kawarabayashi, K. and Jegelka, S., 2018.

31. **Neural Execution of Graph Algorithms**
Velickovic, P., Ying, R., Padovano, M., Hadsell, R. and Blundell, C., 2019.

32. **Graph Theory**
Harary, F., 1969.

33. **A nested-graph model for the representation and manipulation of complex objects**
Poulovassilis, A. and Levene, M., 1994. ACM Transactions on Information Systems, Vol 12(1), pp. 35--68.

34. **Modeling polypharmacy side effects with graph convolutional networks**
Zitnik, M., Agrawal, M. and Leskovec, J., 2018. Bioinformatics, Vol 34(13), pp. i457--i466.

35. **Machine learning in chemical reaction space**
Stocker, S., Csanyi, G., Reuter, K. and Margraf, J.T., 2020. Nat. Commun., Vol 11(1), pp. 5505.

36. **Graphs and Hypergraphs**
Berge, C., 1976. Elsevier.

37. **HyperGCN: A New Method of Training Graph Convolutional Networks on Hypergraphs**
Yadati, N., Nimishakavi, M., Yadav, P., Nitin, V., Louis, A. and Talukdar, P., 2018.

38. **Hierarchical Message-Passing Graph Neural Networks**
Zhong, Z., Li, C. and Pang, J., 2020.

39. **Little Ball of Fur**
Rozemberczki, B., Kiss, O. and Sarkar, R., 2020. Proceedings of the 29th ACM International Conference on Information & Knowledge Management.

40. **Sampling from large graphs**
Leskovec, J. and Faloutsos, C., 2006. Proceedings of the 12th ACM SIGKDD international conference on Knowledge discovery and data mining - KDD '06.

41. **Metropolis Algorithms for Representative Subgraph Sampling**
Hubler, C., Kriegel, H., Borgwardt, K. and Ghahramani, Z., 2008. 2008 Eighth IEEE International Conference on Data Mining.

42. **Cluster-GCN: An Efficient Algorithm for Training Deep and Large Graph Convolutional Networks**
Chiang, W., Liu, X., Si, S., Li, Y., Bengio, S. and Hsieh, C., 2019.

43. **GraphSAINT: Graph Sampling Based Inductive Learning Method**
Zeng, H., Zhou, H., Srivastava, A., Kannan, R. and Prasanna, V., 2019.

44. **How Powerful are Graph Neural Networks?**
Xu, K., Hu, W., Leskovec, J. and Jegelka, S., 2018.

45. **Rep the Set: Neural Networks for Learning Set Representations**
Skianis, K., Nikolentzos, G., Limnios, S. and Vazirgiannis, M., 2019.

46. **Message Passing Networks for Molecules with Tetrahedral Chirality**
Pattanaik, L., Ganea, O., Coley, I., Jensen, K.F., Green, W.H. and Coley, C.W., 2020.

47. **N-Gram Graph: Simple Unsupervised Representation for Graphs, with Applications to Molecules**
Liu, S., Demirel, M.F. and Liang, Y., 2018.

48. **Dual-Primal Graph Convolutional Networks**
Monti, F., Shchur, O., Bojchevski, A., Litany, O., Gunnemann, S. and Bronstein, M.M., 2018.

49. **Viewing matrices & probability as graphs**
Bradley, T..

50. **Graphs and Matrices**
Bapat, R.B., 2014. Springer.

51. **Modern Graph Theory**
Bollobas, B., 2013. Springer Science & Business Media.

52. **Attention Is All You Need**
Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I., 2017.

53. **Graph Attention Networks**
Velickovic, P., Cucurull, G., Casanova, A., Romero, A., Lio, P. and Bengio, Y., 2017.

54. **Set Transformer: A Framework for Attention-based Permutation-Invariant Neural Networks**
Lee, J., Lee, Y., Kim, J., Kosiorek, A.R., Choi, S. and Teh, Y.W., 2018.

55. **Transformers are Graph Neural Networks**
Joshi, C., 2020. NTU Graph Deep Learning Lab.

56. **Using Attribution to Decode Dataset Bias in Neural Network Models for Chemistry**
McCloskey, K., Taly, A., Monti, F., Brenner, M.P. and Colwell, L., 2018.

57. **GNNExplainer: Generating Explanations for Graph Neural Networks**
Ying, Z., Bourgeois, D., You, J., Zitnik, M. and Leskovec, J., 2019. Advances in Neural Information Processing Systems, Vol 32, pp. 9244--9255. Curran Associates, Inc.

58. **Explainability Methods for Graph Convolutional Neural Networks**
Pope, P.E., Kolouri, S., Rostami, M., Martin, C.E. and Hoffmann, H., 2019. 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

59. **Evaluating Attribution for Graph Neural Networks** [HTML]
Sanchez-Lengeling, B., Wei, J., Lee, B., Reif, E., Qian, W., Wang, Y., McCloskey, K.J., Colwell, L. and Wiltschko, A.B., 2020. Advances in Neural Information Processing Systems 33.

60. **Variational Graph Auto-Encoders**
Kipf, T.N. and Welling, M., 2016.

61. **GraphRNN: Generating Realistic Graphs with Deep Auto-regressive Models**
You, J., Ying, R., Ren, X., Hamilton, W.L. and Leskovec, J., 2018.

62. **Optimization of Molecules via Deep Reinforcement Learning**
Zhou, Z., Kearnes, S., Li, L., Zare, R.N. and Riley, P., 2019. Sci. Rep., Vol 9(1), pp. 1--10. Nature Publishing Group.

63. **Self-Referencing Embedded Strings (SELFIES): A 100% robust molecular string representation**
Krenn, M., Hase, F., Nigam, A., Friederich, P. and Aspuru-Guzik, A., 2019.

64. **GraphGen: A Scalable Approach to Domain-agnostic Labeled Graph Generation**
Goyal, N., Jain, H.V. and Ranu, S., 2020.

Updates and Corrections

If you see mistakes or want to suggest changes, please create an issue on GitHub.

Reuse

Diagrams and text are licensed under Creative Commons Attribution CC-BY 4.0 with the source available on GitHub, unless noted otherwise. The figures that have been reused from other sources don't fall under this license and can be recognized by a note in their caption: "Figure from …".

Citation

For attribution in academic contexts, please cite this work as

```
Sanchez-Lengeling, et al., "A Gentle Introduction to Graph Neural Networks", Distill, 2021.
```

BibTeX citation

```
@article{sanchez-lengeling2021a,
  author = {Sanchez-Lengeling, Benjamin and Reif, Emily and Pearce, Adam and Wiltschko, Alexander B.},
  title = {A Gentle Introduction to Graph Neural Networks},
  journal = {Distill},
  year = {2021},
  note = {https://distill.pub/2021/gnn-intro},
  doi = {10.23915/distill.00033}
}
```

Distill is dedicated to clear explanations of machine learning

About Submit Prize Archive RSS GitHub Twitter ISSN 2476-0757